Transportation Research Part E 203 (2025) 104351

Contents lists available at ScienceDirect

Transportation Research Part E

journal homepage: www.elsevier.com/locate/tre

The multi-visit parallel drone scheduling pickup and delivery problem considering multiple trips and time windows

Shanshan Meng *

Business School, Shanghai Jian Qiao University, Shanghai, China
Loughborough Business School, Loughborough University, Leicestershire LE11 3TU, United Kingdom

ARTICLE INFO

Keywords:
Parallel drone scheduling vehicle routing
Pickup and delivery
Time windows
Metaheuristic
Waiting time optimisation

ABSTRACT

We study a new variant of the parallel drone scheduling vehicle routing problem with time windows, where multiple homogeneous trucks and drones operate in parallel to provide parcel pickup and delivery services to customers with the minimum total cost. Additionally, some paired customer requests where each package must be shipped from a pickup customer to another delivery customer by the same vehicle are considered. Each drone has the ability to serve multiple customers in a trip and can make multiple trips to increase utilisation. To better model reality, we propose a large-scale mixed-integer bilinear program that uses a bilinear term to define the energy consumed by drones while hovering at customer locations due to time windows. Later, several strengthening inequalities are added to accelerate the model-solving process. To handle large-size problems, we develop a metaheuristic based on the large neighbourhood search concept with custom destroy and repair operators and complementary local searches, embedding acceleration strategies to avoid infeasible searches. Moreover, a novel algorithm is designed to efficiently optimise the drone waiting times at the depot for each trip, which is key to the problem solution given the constraints of time windows. Finally, various numerical tests and comparisons demonstrate the superior performance of the developed metaheuristic and confirm the effectiveness of strengthening inequalities in model solving. The results highlight the superiority of the parallel mode in terms of cost reduction over the truck-only mode and the advantages of considering drone waiting times and allowing multiple drone visits and trips. Our algorithm outperforms the benchmark algorithms in the literature. The average gaps related to the solution quality from the improved large neighbourhood search are approximately 5%.

1. Introduction

Over the past decade, unmanned aerial vehicles, also known as drones, have been used as part of a new logistic approach to last-mile delivery. A growing number of companies worldwide, such as Amazon and Walmart, are beginning to invest in more serious drone usage. Fortune Business Insights predicts that the commercial drone market will grow from $8 billion in 2022 to $47 billion in 2029 (Flavin, 2022). Using drones for deliveries is considered extremely strategic, as they are lightweight, battery-powered, can fly in approximately straight lines, and are less affected by road traffic conditions. This helps relieve the strain on supply chains caused by increasing e-commerce orders, escalating urban traffic congestion and increasing labour costs (Simmons, 2023), contributing to an efficient, economical and sustainable logistics system. However, drones are generally accompanied by technical defects, such

* Correspondence to: Business School, Shanghai Jian Qiao University, Shanghai, China.
E-mail address: mengss@gench.edu.cn.

https://doi.org/10.1016/j.tre.2025.104351
Received 25 November 2024; Received in revised form 24 June 2025; Accepted 27 July 2025
Available online 8 August 2025
1366-5545/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

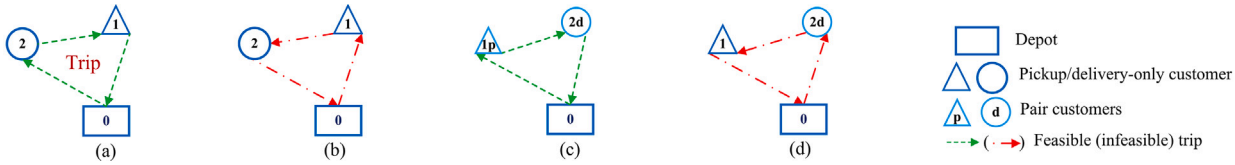

Fig. 1. The same drone trip with different types of customers in opposite directions.

as small load capacities and short flying ranges. Consequently, integrating ground vehicles (referring to trucks) with drones can circumvent the above shortcomings and give rise to a new promising delivery solution. Under this paradigm, trucks can still perform delivery/pickup tasks, but drones can also be deployed to serve other customers. More formally, a drone can take off either from a truck route or from a depot, make a short detour (trip) within its capacity constraints to transport one or more parcels, and then dynamically rejoin a truck or return to the depot for the next trip.

From an academic perspective, inspired by the traditional travelling salesman problem (TSP), Murray and Chu (2015) pioneered a new routing problem with two new prototypical variants: the flying sidekick TSP (FSTSP) and the parallel drone scheduling TSP (PDSTSP). In both cases, a truck and one or more drones cooperate to make deliveries with the objective of minimising completion time, and drones can make a single visit in a trip. However, they differ in the way drones conduct deliveries. For the former, the truck dispatches a drone during its tour to make deliveries (referring to the combined mode). In the latter, the truck and multiple drones execute independent deliveries from a central depot (referring to the parallel mode). Later, Agatz et al. (2018) and Wang et al. (2017) presented the travelling salesman problem with drone (TSPD) and the vehicle routing problem with drones (VRPD), two other new prototype models of the combined mode. Many works have introduced more complicated concepts, such as multiple visits by drones (Meng et al., 2023). However, the parallel mode seems to have received less attention despite its apparent practical applications. Few works have considered such realistic features of the parallel mode, and we have not found any works that explicitly discuss the impact of time windows on pickup and delivery services in the multi-visit parallel drone scheduling vehicle routing problem (PDSVRP).

In addition, the parallel mode has a more realistic application given current technology. Drone operations usually require other equipment in practice, such as self-service delivery lockers. Therefore, it is preferable to plan and operate drones at a fixed location from managerial and safety perspectives, such as a depot. Even if multiple drones operate in parallel, since synchronisation is not needed, seamless service to customers can still be simply achieved, thus ensuring more robustness to the delivery process and effectively increasing usability. Moreover, continuous breakthroughs in drone design and battery technology/endurance also paint a promising picture for drone delivery. For example, DJI introduced a long-distance heavy lifter drone for delivery, i.e., FlyCart 30, which has a 30 kg maximum payload, a 28 km flight distance without a payload, and a 16 km flight distance with a full payload (DJI, 2025). The heavy payload capacity and long flight distance of these drones allow them to ascend beyond traditional logistical limits to deliver a safe, economical, and efficient air transport solution. Technologies that support multiple drone visits have also emerged. For example, the Vulcan UAV Airlift is equipped with a multi-compartment payload, which can be used for multiple deliveries with each flight (Masmoudi et al., 2022).

Some commercial applications of drone pickups have recently been piloted as well. One example is Wing's new system, which enables its drones to autonomously pick up packages loaded onto a pickup station and take pickup and delivery jobs without having to immediately return to the origin point (Shakir, 2023; Wing, 2025). As another example, the logistics company Zipline has worked on food delivery, offering error-proof loading and pickup services from restaurants to customers in under 10 min without traffic, stoplights, or detours (Zipline, 2025).

To this end, we introduce an innovative, parallel pickup and delivery problem characterised by the multi-visit capability of drones and time windows, i.e., the so-called multi-visit parallel drone scheduling pickup and delivery problem considering multiple trips and time windows (mPDPD-mT), with consideration of the optimal waiting times for drones, an important lever to consider in such problems but often neglected. The optimisation objective is to minimise the total cost incurred by both types of vehicles. Our study differs from previous works in that (i) we investigate a new variant of the pickup and delivery problem where drones can conduct multiple visit trips from a central depot multiple times. (ii) We consider a more realistic energy consumption model for drones, including the energy consumed by drones while hovering because of their early arrival at customer locations. (iii) Each customer has a time window within which the service can begin.

Furthermore, three types of services are included, namely, pickup only, delivery only and one-to-one pickup and delivery (pair service). For each customer pair, the pickup pair must be served before its counterpart because of their precedence constraints. Therefore, each route in the mPDPD-mT is asymmetric. An example is shown in Fig. 1, where the drone's carrying capacity is 3, and the demand for customers 1 and 2 is 2 each. As shown in Fig. 1a and b, the same trip in opposite directions may consume different amounts of energy or even lead to infeasible service, and whether two customers in the same trip are paired or not, the variation in payload is significantly different (Fig. 1c and d).

Moreover, previous related studies have generally assumed that a drone's battery duration is fixed and that consumption is proportional to only the flying distance or time. However, the energy consumption of drones is affected by several key factors, such as payload, and involves different manoeuvres, such as hover energy consumption at customer locations due to time window constraints. Therefore, drone waiting time at the depot is important to consider, as it directly affects trip feasibility and hover energy

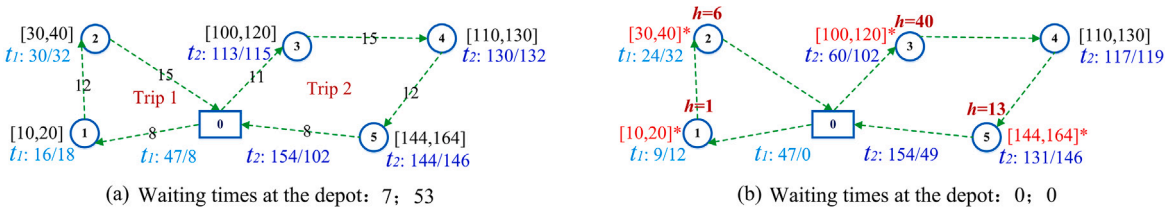

Fig. 2. The same drone trips with different waiting times.

consumption of the next trip. Inappropriate waiting times in practice may cause drones to miss service time (late arrival) or hover for too long (early arrival), leading to service failure or even safety hazards, thereby forcing seamless services to be suspended. Fig. 2 illustrates the above challenges with an example, where the times for the drone to arrive and leave each node in trips 1 and 2 are denoted by '$t1$' and '$t2$', respectively, and the time windows (appearing in square brackets) and travel times are marked next to the nodes and on the arcs, respectively. The time to launch or retrieve a drone is 1, and the time taken by drones to serve a customer is 2. As shown in Fig. 2a, with waiting time optimisation, the drone arrives at all customer locations within their time windows (zero hover time). Otherwise, the hover energy consumption increases significantly as the drone arrives earlier at four customer locations (the corresponding hover times h are marked) (see Fig. 2b). Moreover, both the payload and duration of hovering depend on routing variables in mPDPD-mT and are unknown a priori, resulting in complicated calculations.

To our knowledge, this is the first study in the literature to address these challenges. Most previous studies related to the parallel mode adhered to single-visit and delivery-only drone services (Saleu et al., 2022; Nguyen et al., 2022), and few considered integrating pickup and delivery in the multi-visit PDSVRP context. One such work by Ham (2018) expanded the basic PDSTSP to pickup and delivery services, that is, allowing a truck. Additionally, it was assumed that the drone has a single capacity and did not consider either the pair service or the variable battery endurance of the drones. The latest work that explicitly considered pickup and delivery was that of Meng et al. (2024). They provided a novel VRPD that considers time windows and assumes that each truck and drone works in pairs and that drones are operated by trucks from a set of dedicated stations. In Section 2, we provide a comprehensive review of related works.

In summary, this work contributes to the literature as follows. (i) We introduce a new PDSVRP variant, which considers three types of services with time windows and encourages multiple drone visits and trips. A new mixed-integer bilinear program (MIBLP) is then mathematically proposed to formulate this problem, incorporating problem-tailored strengthening inequalities to cut off infeasible routing. With the exception that the energy consumed by drones while hovering is bilinear, the objective function and other constraints are linear. (ii) As it is more complicated than the already NP-hard problem, VRP, a metaheuristic based on the large neighbourhood search principle, is introduced to find solutions for large-size instances. Moreover, an efficient method with enhanced theoretical cuts is developed to determine the waiting times of drones at the depot. (iii) Extensive numerical experiments are conducted to investigate the efficacy of the designed metaheuristic, as well as the remarkable benefits of the parallel mode and considering drone waiting times. The results also show the impacts of the most relevant parameters on the output and provide managerial insights for better managing the parallel transport system. This research studies the operational challenges of the parallel mode for pickup and delivery, which has high significance in practice and wide applicability in numerous logistic systems. It offers a new logistics solution for popular take-out and instant delivery, as well as a new attempt to manage returns or raw material pickups during the delivery process, which is increasingly faced by industry practice.

In the remainder of the paper, we continue with the review of related works in Section 2. Then Section 3 formally defines our problem, illustrates the mathematical model, together with strengthening inequalities. Section 4 detailedly describes our solution approaches. Computational results are displayed in Section 5, while conclusions are drawn in Section 6.

2. Literature review

As mentioned above, Murray and Chu (2015) first introduced the basic PDSTSP, in which a single truck and one or more identical drones operate in parallel to perform deliveries, with the objective of minimising the completion time. The single truck delivers parcels along a classic TSP route, and drones fly back and forth between the depot and customers, performing single-visit trips. Then, they formulate the problem as a mixed-integer linear program (MILP) and provide a greedy heuristic to guide the assignment of customers to the truck or drones. Note that synchronisation between the truck and drones is not required here. However, they assume an infinite capacity of the truck and do not consider either customer pickup demands or time windows.

Some researchers have studied the above PDSTSP. To reach a solution, Mbiadou Saleu et al. (2018) proposed an iterative two-step heuristic by first partitioning customers and then optimising routes. Dell'Amico et al. (2020) presented a fast heuristic and a random restart local search. Dinh et al. (2021) develop a hybrid ant colony optimisation (HACO) metaheuristic in which three main components were incorporated. Lei and Chen (2022) improved the variable neighbourhood search (IVNS) by using a shaking procedure and an adaptive mechanism to optimise the initial solution. Recently, Chen and Wang (2024) presented a ruin-and-recreate(R&R)-based metaheuristic enhanced by the Q-learning technique. Moreover, Nguyen et al. (2023) deployed an exact algorithm, the branch-and-cut (B&C) algorithm, in which a faster procedure was designed to separate constraints. More complicated

scenarios was explored by Nguyen and Hà (2023), who extended the PDSTSP with collective drones to improve drone utilisation, particularly for heavy item deliveries. That is, they coupled multiple drones to a so-called 'collective drone' (c-drone) so that it could transport heavy goods. Moreover, a tailored R&R metaheuristic, as well as a topological-sort-based evaluation procedure, was proposed.

Some studies have extended the PDSTSP by enabling multiple trucks. In particular, Saleu et al. (2022) solved a parallel drone scheduling multiple TSPs via an iterated local search (ILS)-based metaheuristic and a simple B&C approach. Later, Montemanni et al. (2024) extended the above c-drone paradigm to multi-truck scenario and contributed two constraint programs and a mixed-integer program. Moreover, Nguyen et al. (2022) studied the travel cost of the PDSVRP and improved the R&R algorithm to solve large-size problems. Montemanni and Dell'Amico (2023) studied both the completion time and operating cost of the PDSVRP and addressed these two problems with different constraint programming models. Wang et al. (2019) considered a more flexible delivery scenario that simultaneously employed three types of deliveries either by independent trucks or drones, or by drones carried by trucks, to achieve a mixed logistics system with high efficiency. They proposed a heuristic algorithm that included three steps: customer clustering, flight construction and global reoptimisation. Nevertheless, as noted above, their studies included more realistic factors and scenarios, such as involving multiple trucks, cost optimisation, and flexible delivery modes, but they focused only on single drone visits and delivery-only scenarios and did not include the context of our problem, especially with respect to the time window concerned and the resulting hover energy consumption optimisation and waiting time decisions.

Some works on parallel modes have involved time windows. For example, Ham (2018) considered that drones can also perform pickup tasks in addition to dropping into the PDSTSP and modelled the problem in a constraint programming form, which was then tested on various instances. However, the author assumed a single-unit capacity of drones and an infinite capacity of trucks and neglected hover energy consumption due to time windows and induced waiting times. Chen et al. (2022) addressed a dynamic same-day delivery problem in which multiple heterogeneous vehicles and drones independently delivered goods to customers ahead of their service deadlines. Moreover, a deep Q-learning approach was adopted to quickly decide whether to assign a request to a drone or a truck or not to provide service at all. Similarly, they continued to assume single drone visit and provided only partial services to maximise the reward.

As mentioned above, the above PDSTSP/PDSVRP works focused mainly on delivery services, and drones usually can make a single visit, a few of which involve pickup services and time windows. A few works have investigated pickup and delivery issues, but trucks and drones operate in a coordinated paradigm (Meng et al., 2023). For example, Karak and Abdelghany (2019) considered a single truck driving between a set of available stations from which drones were dispatched for final services and improved the Clarke and Wright (C&W) algorithm to minimise the total travel cost. The work by Luo et al. (2022) investigated a combined routing problem that focused on providing only pair services in the VRPD context and designed an ILS algorithm to minimise the total operational cost for both types of vehicles. In contrast to the above works, they considered the payload-based energy consumption of drones but again did not include time windows. Moreover, Meng et al. (2023) and Jiang et al. (2024) improved the simulated annealing algorithm (ISA) and adaptive large neighbourhood search (ALNS) for a combined pickup and delivery problem with multi-visit drones. They allowed multiple trucks but, similar to Karak and Abdelghany (2019), did not consider either pair services or time windows. Li et al. (2024) examined a similar problem but emphasised that each truck must wait at each launch node from where drones depart and return and that all delivery tasks must be executed, whereas the pickup tasks can be partially performed with certain rewards. In addition, they solved the challenging problem by devising a tailored exact algorithm enhanced by cutting inequalities and heuristics for pricing. More recently, Meng et al. (2024) provided a complicated pickup and delivery system and developed a custom ALNS for problem solving. They considered a set of available stations to assist drone operations and included both pair services and time windows. Nevertheless, our problem focuses on the parallel mode and considers operating multiple drone trips and waiting time decisions from the depot.

On the literature related to parallel pickup and delivery, we can also mention the heterogeneous vehicle routing problem with simultaneous pickup and delivery (HVRPSPD), which considers different types of available vehicles to serve customers with both pickup and delivery requests. For the solution, Avci and Topaloglu (2016) developed an adaptive hybrid local search algorithm (HLS), which integrated a nonmonotone threshold function in tabu search and could solve random instances with up to 550 customers. Later, Nepomuceno et al. (2019) solved the problem with a fast algorithm, in which a node was selected probabilistically from the nearest neighbour or at random. The experimental results of the benchmark instances showed that their algorithm could provide competitive results for larger instances with a relatively short time. Then Keçeci et al. (2021) formulated the problem into a polynomial-size mixed integer program (MIP) and developed a hybrid simulated annealing algorithm with local search (SA-LS) for large-size problems.

A variant of HVRPSPD with time windows was considered by Qu and Bard (2015), who assumed that the vehicles could be configured in advance to handle various types of requests, then developed a branch and price and cut based method, and tested it both on real and random instances containing 50 customers. Madankumar and Rajendran (2019) modelled the problem as an efficient MILP via node-specific variables to reduce the number of binary variables. Experimental results demonstrated the superior performance of their MILP in providing tighter lower bounds, as well as computational time and solution quality compared to other models in the literature.

As described, our problem shares some similarities with the above HVRPSPDs, but they are still very different. The most studied HVRPSPD and its variants generally aim at solving the one-to-many-to-one pickup and delivery problems in the context of reverse logistics (Koç et al., 2020), seldom consider pair services. In addition, HVRPSPDs only consider the differences in vehicle carrying capacity and each vehicle can be assigned at most once, as assumed in VRP, do not involve multiple trips and drone characteristics, such as faster speed, limited flying distance, energy consumption related to payload, and hover energy consumption due to early arrival.

In summary, the above review concludes that a new PDSVRP variant has not yet been studied. The majority of current works have focused on addressing the basic PDSTSP/PDSVRP, which considers delivery-only issues with single capacity of drones. A few works involving pickup and delivery assumed that trucks and drones served customers in tandem and seldom considered customer time windows; thus, the corresponding approach is not yet available. As highlighted above, we extend the extant literature by presenting a multi-visit parallel routing system that considers both pickup and delivery as well as customer time windows. Moreover, hover energy consumption and waiting time decisions induced by time windows are included. We formulate the new problem as an MIBLP, using a bilinear term to capture the hover energy of drones, enhanced by strengthening inequalities. To solve large-size problems, we develop an effective algorithm based on large neighbourhood search (LNS) principle to find near-optimal solutions to the routing and timing of trucks and drones in parallel, in which an efficient method with enhanced theoretical cuts is used to determine waiting times. In Table 1, we present a brief summary of the most related works.

3. Problem description and formulation

3.1. Problem description

In the mPDPD-mT, multiple homogeneous trucks and drones are available to work in parallel, starting and returning from the same depot to serve customers, each with a pickup-only, delivery-only or one-to-one pickup or delivery request. Each customer can be visited by a truck or drone exactly once and has a time window within which the service can be started. Some customers are in pairs, where a pickup customer must be visited before a corresponding delivery, and each pair is assumed to be served by the same vehicle (Luo et al., 2022). However, some customers, for practical reasons such as excessively heavy packages, cannot be assigned to drones (truck-only customers). Each drone can deliver or pick up multiple packages in a trip within its load and battery capacities. If a drone arrives at a customer location before the time window, it has to hover and consumes additional energy until the time window opens. Notably, each truck can travel one trip at most, whereas each drone can make multiple trips. Once a drone's battery is replaced at the depot, it is ready for the next trip.

The objective is to minimise the total cost of service for all customers by deploying optimal numbers of both types of vehicles with optimal routes or trips. The total cost is calculated by the variable and fixed costs of deploying both types of vehicles, where the variable costs for trucks and drones are measured by the distance travelled by trucks and the energy consumed by drones, respectively. For practical purposes, we measure the distances travelled by trucks and drones with the Manhattan and Euclidean metrics, respectively. Fig. 3 depicts a visual illustration of a feasible solution for an example involving 17 customers served by 2 trucks and 3 drones. Overall, the following assumptions are used throughout our study for more practical applications:

- Each drone can carry multiple parcels in a trip and can make multiple trips, starting and ending at the depot.
- Each customer has a pickup-only, delivery-only or one-to-one pickup or delivery request. Two paired requests are served by the same vehicle and the pickup request must be visited before its counterpart.
- Both types of vehicles have limited load capacity, and the energy consumption of drones is subject to battery capacity and is proportional to load and duration.
- Times to serve customers and launch and retrieve drones are needed.
- Service for each customer must start within the time window. Therefore, a drone must hover and consume additional energy if it arrives at a customer location before the time window.
- To reduce possible hover time, drones are allowed to wait at the depot before starting trips.

In the mPDPD-mT, we attempt to integrate more practical aspects of parallel pickup and delivery (with fewer restrictions on operations) for real-life applications, such as multiple drone visits and trips, three kinds of services, and time windows, as well as induced hover energy consumption and waiting time decisions.

3.2. Notation

Given a digraph $G = (N, A)$, with N and A being the node and arc sets, respectively. For customer pairs, let $N_{pc} = \{1, \dots, P\}$ and $N_{dc} = \{P+1, \dots, 2P\}$ be the sets of the pickup pairs and their delivery counterparts, where P denotes the number of pairs. Therefore, when a pickup one indexed by $i \in N_{pc}$, its corresponding delivery is $i+P$. Remaining customers are non-pair, namely pickup only and delivery only, represented by N_{po} and N_{do}, respectively. Therefore, we have all customer set $N_c = N_{pc} \cup N_{dc} \cup N_{po} \cup N_{do} = \{1, \dots, n\}$ and all node set $N = \{0\} \cup N_c \cup \{n+1\}$, where 0 and $n+1$ represent the starting and ending of each route. Denote by N_t and N_d the truck-only and drone-eligible customer sets, and $N_o = \{0\} \cup N_c$ and $N_e = N_c \cup \{n+1\}$ the sets of outbound and inbound nodes, respectively. Thus the all arc set $A = \{(i,j) | i \in N_o, j \in N_e, i \neq j\}$. Moreover, we use $A_d = \{(i,j) | i \in \{0\} \cup N_d, j \in N_d \cup \{n+1\}, i \neq j\}$ to denote the set of arcs that drones can fly over. Denote by K_t and K_d the sets of available trucks and drones, respectively. We

Table 1
Summary of the most related works.

Reference	Syn.	Multi-capacity	Multi-departure	#T	#D	Time window	Service type	Hover energy	Waiting times	Objective	Solution method
Murray and Chu (2015)	×	×	√	1	n	×	D	×	×	Min time	MILP, greedy heuristic
Mbiadou Saleu et al. (2018)	×	×	√	1	n	×	D	×	×	Min time	Iterative two-step heuristic
Dell'Amico et al. (2020)	×	×	√	1	n	×	D	×	×	Min time	MILP, random restart local search
Dinh et al. (2021)	×	×	√	1	n	×	D	×	×	Min time	Metaheuristic (HACO)
Lei and Chen (2022)	×	×	√	1	n	×	D	×	×	Min time	Metaheuristic (IVNS)
Chen and Wang (2024)	×	×	√	1	n	×	D	×	×	Min time	Q-learning, metaheuristic (R&R)
Nguyen et al. (2023)	×	×	√	1	n	×	D	×	×	Min time	Branch-and-cut
Nguyen and Hà (2023)	×	×	√	1	n	×	D	×	×	Min time	MILP, metaheuristic (R&R)
Saleu et al. (2022)	×	×	√	m	n	×	D	×	×	Min time	MILP, metaheuristic (ILS)
Montemanni et al. (2024)	×	×	√	m	n	×	D	×	×	Min time	Constraint programming
Nguyen et al. (2022)	×	×	√	m	n	×	D	×	×	Min travel cost	MILP, metaheuristic (R&R)
Montemanni and Dell'Amico (2023)	×	×	√	m	n	×	D	×	×	Min time and cost	Constraint programming
Wang et al. (2019)	Both	√	×	m	n	×	D	×	×	Min time	Three-step heuristic
Ham (2018)	×	×	√	m	n	√	P+D	×	×	Min time	Constraint programming
Chen et al. (2022)	×	×	√	m	n	√	D	×	×	Max reward	Deep Q-learning
Karak and Abdelghany (2019)	√	√	×	1	n	×	P+D	×	×	Min travel cost	MIP, improved C&W
Li et al. (2024)	√	√	×	m	n	√	P+D	×	×	Min (cost-revenue)	Branch-and-cut
Luo et al. (2022)	√	√	×	m	m	×	O	×	×	Min operational cost	Metaheuristic (ILS)
Meng et al. (2023)	√	√	×	m	m	×	P+D	×	×	Min total cost	MILP, metaheuristic (ISA)
Jiang et al. (2024)	√	√	×	m	n	×	P+D	×	×	Min travel cost	MILP, metaheuristic (ALNS)
Meng et al. (2024)	√	√	×	m	m	√	P+D+O	√	T	Min total cost	MIBLP, metaheuristic (ALNS)
This paper	×	√	√	m	n	√	P+D+O	√	D	Min total cost	MIBLP, metaheuristic (ILNS)

Syn.: two types of vehicles work in tandem. Multi-capacity: drones can carry multiple parcels. Multi-departure: drones can depart from the depot multiple times. #T: number of trucks. #D: number of drones. Time window: time window is considered. Service type: pickup (P), delivery (D) and one-to-one pickup and delivery (O). Hover energy: hover energy consumption of drones due to time windows is considered. Waiting times: waiting time decision is considered (T: truck, D: drone).

define the time window for the depot as $[0, l_{max}]$, where l_{max} denotes the latest time after which all vehicles cannot return to the depot.

In summary, all the sets, parameters and variables are listed in Table 2. The units of parameters involving weight, time, energy, distance, cost and speed are set as kg, min, Wh, km, $ and km/h, respectively. Moreover, for some notations used by both trucks and drones, we add *hat* to the same notations for distinguishing drones. Note that variable $Z_p^{k,f}$ is used to indicate whether two paired customers are served within a trip; more details are provided in Section 3.3.5.

3.3. The model

Considering pair service and time windows, in addition to a more realistic drone energy consumption model and multi-trip operations, the mPDPD-mT is much more complex than the PDSVRP. In what follows, we develop an MIBLP to formulate the problem, in which, except for the constraint on the hover energy consumption of drones, which is bilinear, all other parts of the model are linear. Section 3.3.1 describes the objective function, then Section 3.3.2 introduces the routing constraints, followed by the timing constraints documented in Section 3.3.3. In Sections 3.3.4 and 3.3.5, we give the constraints on the energy consumption of drones and the load capacity of the two types of vehicles, respectively.

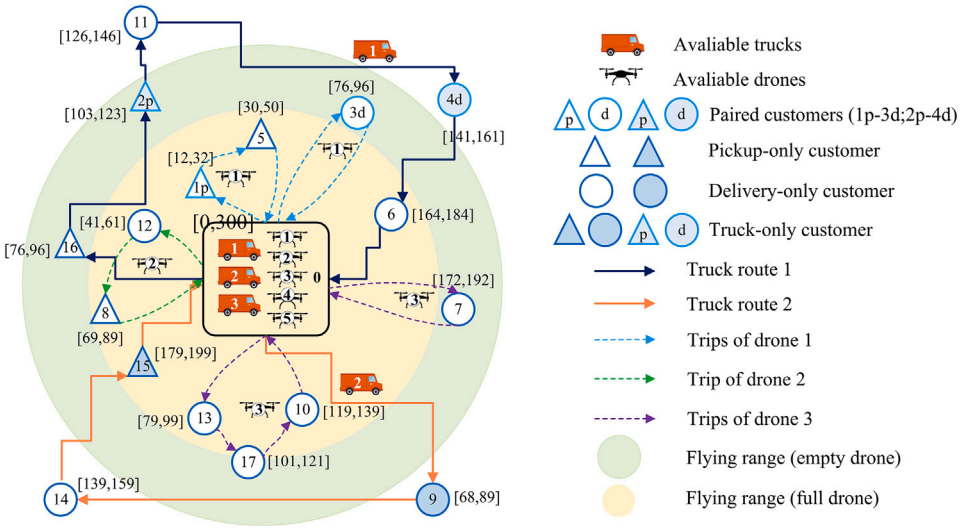

Fig. 3. An illustrative example of the parallel system.

3.3.1. Objective function

$$min \quad c \sum_{k \in K_t} \sum_{(i,j) \in A} x_{ij}^k r_{ij} + \hat{c} \sum_{k \in K_d} \sum_{f \in F} \left(\sum_{(i,j) \in A_d} \varepsilon_{ij}^{k,f} + \sum_{i \in N_d} (\tilde{\varepsilon}_i^{k,f} + \bar{\varepsilon}_i^{k,f}) \right) + c^0 \sum_{k \in K_t} u^k + \hat{c}^0 \sum_{k \in K_d} \hat{u}^k \tag{1}$$

The objective function (1) minimises the total cost including the travel cost of trucks, total energy cost incurred by drones (the sum of flying, serving and hover energy costs of drones), and fixed costs of deploying both types of vehicles.

3.3.2. Routing constraints

$$\sum_{i \in N_e} x_{0i}^k = \sum_{i \in N_o} x_{i,n+1}^k = 1, k \in K_t \tag{2a}$$

$$\sum_{i \in N_d \cup \{n+1\}} y_{0i}^{k,f} = \sum_{i \in \{0\} \cup N_d} y_{i,n+1}^{k,f} = 1, f \in F, k \in K_d \tag{2b}$$

$$\sum_{i \in N_o} x_{ij}^k = \sum_{i \in N_e} x_{ji}^k, j \in N_c, k \in K_t \tag{3a}$$

$$\sum_{i \in \{0\} \cup N_d} y_{ij}^{k,f} = \sum_{i \in N_d \cup \{n+1\}} y_{ji}^{k,f}, j \in N_d, f \in F, k \in K_d \tag{3b}$$

$$\sum_{i \in N_o} x_{ij}^k = \sum_{i \in N_c} x_{i,j+P}^k, j \in N_{pc}, k \in K_t \tag{4a}$$

$$\sum_{f \in F} \sum_{i \in \{0\} \cup N_d} y_{ij}^{k,f} = \sum_{f \in F} \sum_{i \in \{0\} \cup N_d} y_{i,j+P}^{k,f}, j \in N_{pc} \cap N_d, k \in K_d \tag{4b}$$

$$\sum_{k \in K_t} \sum_{i \in N_o} x_{ij}^k + \sum_{k \in K_d} \sum_{f \in F} \sum_{i \in \{0\} \cup N_d} y_{ij}^{k,f} = 1, j \in N_c \tag{5a}$$

$$\sum_{k \in K_t} \sum_{i \in N_o} x_{ij}^k = 1, j \in N_t \tag{5b}$$

Table 2
Notation list.

Sets	
N_{pc}	Pickup pair set, $N_{pc} = \{1, \dots, P\}$
N_{dc}	Delivery pair set, $N_{dc} = \{P+1, \dots, 2P\}$
N_{po}	Pickup only set
N_{do}	Delivery only set
N_c	All customer set, $N_c = N_{pc} \cup N_{dc} \cup N_{po} \cup N_{do} = \{1, 2, \dots, n\}$
N	All node set, $N = \{0\} \cup N_c \cup \{n+1\}$
N_o	Outbound node set, $N_o = \{0\} \cup N_c$
N_e	Inbound node set, $N_e = N_c \cup \{n+1\}$
N_t	Truck-only set
N_d	Drone-eligible set
A	All arc set, where $A = \{(i,j) \mid i \in N_o, j \in N_e, i \neq j\}$
A_d	Set for arcs that can be flown by drones independently, $A_d = \{(i,j) \mid i \in \{0\} \cup N_d, j \in N_d \cup \{n+1\}, i \neq j\}$
K_t	Available truck (indexed by k) set
K_d	Available drone (indexed by k) set
F	Trip (indexed by f) set of each drone
Parameters	
w_0	Weight of each no-load drone, including the mass of drone and battery
λ	Energy that a drone consumes per unit weight per unit time
E	Drone's battery capacity
Q, $\hat{Q}$	Carrying capacity of each type of vehicle
v, $\hat{v}$	Constant speed for each type of vehicle
c^0, $\hat{c}^0$	Fixed cost of each type of vehicle being deployed
c	Truck travel cost per unit distance
$\hat{c}$	Drone energy cost per unit energy
r_{ij}	Travel distance of each truck from i to j
$\hat{r}_{ij}$	Travel distance of each drone from i to j
d_i	Customer demand. For a pick-up customer, $d_i < 0$, and for a delivery customer, $d_i > 0$
$[e_i, l_i]$	Time window of i
s_i	Time consumed to serve i
S_l	Time consumed to launch a drone (including cargo handling)
S_r	Time consumed to retrieve a drone (including cargo handling and battery replacement)
B	A big enough number
Decision variables	
$x_{ij}^k \in \{0,1\}$	$x_{ij}^k = 1$ indicates that truck $k \in K_t$ travels from $i \in N_o$ to $j \in N_e$ and otherwise $x_{ij}^k = 0$
$y_{ij}^{k,f} \in \{0,1\}$	$y_{ij}^{k,f} = 1$ indicates that drone $k \in K_d$ flies from $i \in \{0\} \cup N_d$ to $j \in N_d \cup \{n+1\}$ in trip f and otherwise $y_{ij}^{k,f} = 0$
$\theta^{k,f} \in [0, l_{max}]$	Duration of drone $k \in K_d$ waiting at the depot in trip f
Auxiliary variables	
$w_i^k \in [0, Q]$	Quantities carried by truck $k \in K_t$ when leaving $i \in N_o$
$\hat{w}_i^{k,f} \in [0, \hat{Q}]$	Quantities carried by drone $k \in K_d$ when leaving $i \in \{0\} \cup N_d$ in trip f
$\epsilon_i^{k,f} \in [0, E]$	Total energy that drone $k \in K_d$ has consumed when leaving $i \in \{0\} \cup N_d$ in trip f
$\epsilon_{ij}^{k,f} \in [0, E]$	Amount of energy drone $k \in K_d$ consumes to traverse arc $(i,j) \in A_d$ in trip f
$\bar{\epsilon}_i^{k,f} \in [0, E]$	Amount of energy drone $k \in K_d$ consumes during serving $i \in N_d$ in trip f
$\bar{\epsilon}_i^{k,f} \in [0, E]$	Amount of energy drone $k \in K_d$ consumes during hovering at $i \in N_d$ in trip f
$h_i^{k,f} \in [0, E/(\lambda \cdot w_0)]$	Hovering duration of drone $k \in K_d$ at $i \in N_d$ in trip f, $h_i^{k,f} = max\{0, e_i - \hat{\tau}_i^{k,f}\}$
$\tau_i^k \in [0, l_{max}]$	Time when truck $k \in K_t$ arrives at $i \in N_e$
$\hat{\tau}_i^{k,f} \in [0, l_{max}]$	Time when drone $k \in K_d$ arrives at $i \in N_d \cup \{n+1\}$ in trip f
$v_i \in [e_i, l_i]$	Service start time of node $i \in N_c$ either by trucks or drones
$u^k \in \{0,1\}$	$u^k = 1$ indicates that truck $k \in K_t$ is deployed and otherwise $u^k = 0$
$\hat{u}^k \in \{0,1\}$	$\hat{u}^k = 1$ indicates that drone $k \in K_d$ is deployed and otherwise $\hat{u}^k = 0$
$Z_p^{k,f} \in \{0,1\}$	$Z_p^{k,f} = 1$ indicates that a delivery-pair node $p \in N_{dc}$ is served but its counterpart is not served by trip f of drone $k \in K_d$ and otherwise $Z_p^{k,f} = 0$
$\gamma_i \in \{0,1\}$	$\gamma_i = 1$ indicates the time when the service vehicle arrives at i is earlier than the time window and otherwise $\gamma_i = 0$

$$\sum_{i \in N_d} y_{0i}^{k,f+1} \leq \sum_{i \in N_d} y_{i,n+1}^{k,f}, f \in F \setminus |F|, k \in K_d \tag{6}$$

$$\sum_{i \in N_o} \sum_{j \in N_c} x_{ij}^k \leq n \cdot u^k, k \in K_t \tag{7a}$$

$$\sum_{f \in F} \sum_{i \in \{0\} \cup N_d} \sum_{j \in N_d} y_{ij}^{k,f} \leq n \cdot \hat{u}^k, k \in K_d \tag{7b}$$

Constraints (2a) and (2b) ensure that each truck/drone trip starts and returns from the depot just once. Constraints (3a) and (3b) ensure the flow conservation for each vehicle at a customer location. Constraints (4a) and (4b) restrict that the service of two paired customers is performed by the same vehicle. Constraints (5a) and (5b) guarantee all customers are served and each only once by a truck or drone, while truck-only customers must be assigned to trucks. Constraint (6) states that a drone can leave the depot again only after it has returned. Constraints (7a) and (7b) state that if a truck/drone serves customers, it is deployed.

3.3.3. Timing constraints

$$v_i + s_i \le v_{i+P}, i \in N_{pc} \tag{8}$$

$$v_i \le e_i + B(1-\gamma_i), i \in N_c \tag{9a}$$

$$\tau_i^k - B(1 - \sum_{j\in N_o} x_{ji}^k) \le v_i \le \tau_i^k + B(1 - \sum_{j\in N_o} x_{ji}^k + \gamma_i), i \in N_c, k \in K_t \tag{9b}$$

$$\hat{\tau}_i^{k,f} - B(1 - \sum_{j\in\{0\}\cup N_d} y_{ji}^{k,f}) \le v_i \le \hat{\tau}_i^{k,f} + B(1 - \sum_{j\in\{0\}\cup N_d} y_{ji}^{k,f} + \gamma_i), i \in N_d, f \in F, k \in K_d \tag{9c}$$

$$\tau_i^k \ge e_i - B(1 - \sum_{j\in N_o} x_{ji}^k + \gamma_i), \tau_i^k \le e_i + B(2 - \sum_{j\in N_o} x_{ji}^k - \gamma_i), i \in N_c, k \in K_t \tag{10a}$$

$$\hat{\tau}_i^{k,f} \ge e_i - B(1 - \sum_{j\in\{0\}\cup N_d} y_{ji}^{k,f} + \gamma_i), \hat{\tau}_i^{k,f} \le e_i + B(2 - \sum_{j\in\{0\}\cup N_d} y_{ji}^{k,f} - \gamma_i), i \in N_d, f \in F, k \in K_d \tag{10b}$$

$$\frac{r_{0i}}{v} - B(1 - x_{0i}^k) \le \tau_i^k \le \frac{r_{0i}}{v} + B(1 - x_{0i}^k), i \in N_c, k \in K_t \tag{11a}$$

$$S_l + \theta^{k,0} + \frac{\hat{r}_{0i}}{\hat{v}} - B(1 - y_{0i}^{k,0}) \le \hat{\tau}_i^{k,1} \le S_l + \theta^{k,0} + \frac{\hat{r}_{0i}}{\hat{v}} + B(1 - y_{0i}^{k,0}), i \in N_d, k \in K_d \tag{11b}$$

$$\begin{aligned} &\hat{\tau}_{n+1}^{k,f-1} + S_r + \theta^{k,f} + S_l + \frac{\hat{r}_{0i}}{\hat{v}} - B(1 - y_{0i}^{k,f}) \le \hat{\tau}_i^{k,f} \\ &\le \hat{\tau}_{n+1}^{k,f-1} + S_r + \theta^{k,f} + S_l + \frac{\hat{r}_{0i}}{\hat{v}} + B(1 - y_{0i}^{k,f}), i \in N_d, f \in F \setminus \{0\}, k \in K_d \end{aligned} \tag{11c}$$

$$v_i + s_i + \frac{r_{ij}}{v} - B(1 - x_{ij}^k) \le \tau_j^k \le v_i + s_i + \frac{r_{ij}}{v} + B(1 - x_{ij}^k), i \in N_c, j \in N_e, k \in K_t \tag{12a}$$

$$\begin{aligned} &v_i + s_i + \frac{\hat{r}_{ij}}{\hat{v}} - B(1 - y_{ij}^{k,f}) \le \hat{\tau}_j^{k,f} \\ &\le v_i + s_i + \frac{\hat{r}_{ij}}{\hat{v}} + B(1 - y_{ij}^{k,f}), i \in N_d, j \in N_d \cup \{n+1\}, f \in F, k \in K_d \end{aligned} \tag{12b}$$

Constraint (8) specifies that each pickup-pair node must be visited before its delivery counterpart. Constraint (9a) states that if the arrival time of a vehicle at each node is earlier than the time window, it must wait and commence service until the time window opens. Otherwise, the service starts when the vehicle just arrives, as shown in Constraints (9b) and (9c). Constraints (10a) and (10b) define γ_i. Constraints (11a) and (11c) specify the times when a truck and a drone arrive at each node from the depot. Note that Constraints (11b) and (11c) are specifically defined for the first and follow-up trips of each drone, respectively. The arrival times of a truck and a drone at the next node from a customer are calculated in Constraints (12a) and (12b), respectively.

3.3.4. Energy consumption constraints of drones

Considering the important role of load and time on drone energy consumption, the payload-duration-based model in Meng et al. (2024) is adopted to calculate drone energy consumption. In a trip f of drone k, the energy consumed to fly over arc (i,j) is defined as $\lambda \cdot (w_0 + \hat{w}_i^{k,f}) \cdot \frac{\hat{r}_{ij}}{\hat{v}}$, energy consumed during serving a customer i is defined as $\lambda \cdot (w_0 + \hat{w}_i^{k,f} + d_i) \cdot s_i$, and energy consumed to hover at a node i is defined as $\lambda \cdot (w_0 + \hat{w}_i^{k,f} + d_i) \cdot h_i^{k,f}$.

Then, the energy consumption of drones is subject to

$$0 \le \varepsilon_{ij}^{k,f} \le E \cdot y_{ij}^{k,f}, (i,j) \in A_d, f \in F, k \in K_d \tag{13a}$$

$$\lambda(w_0 + \hat{w}_i^{k,f})\frac{\hat{r}_{ij}}{\hat{v}} - B(1 - y_{ij}^{k,f}) \le \varepsilon_{ij}^{k,f} \le \lambda(w_0 + \hat{w}_i^{k,f})\frac{\hat{r}_{ij}}{\hat{v}}, (i,j) \in A_d, f \in F, k \in K_d \tag{13b}$$

$$0 \le h_i^{k,f} \le B \cdot \gamma_i, i \in N_d, f \in F, k \in K_d \tag{14a}$$

$$\varepsilon_i - \hat{\tau}_i^{k,f} \le h_i^{k,f} \le \varepsilon_i - \hat{\tau}_i^{k,f} + B(1-\gamma_i), i \in N_d, f \in F, k \in K_d \tag{14b}$$

$$0 \le \tilde{\varepsilon}_i^{k,f} + \bar{\varepsilon}_i^{k,f} \le B \sum_{j \in \{0\} \cup N_d} y_{ji}^{k,f}, i \in N_d, f \in F, k \in K_d \tag{15a}$$

$$\lambda(w_0 + \hat{w}_i^{k,f} + d_i)(s_i + h_i^{k,f}) - B(1 - \sum_{j \in \{0\} \cup N_d} y_{ji}^{k,f}) \le \tilde{\varepsilon}_i^{k,f} + \bar{\varepsilon}_i^{k,f}$$
$$\le \lambda(w_0 + \hat{w}_i^{k,f} + d_i)(s_i + h_i^{k,f}), i \in N_d, f \in F, k \in K_d \tag{15b}$$

$$0 \le \varepsilon_0^{k,f} \le B(1 - \sum_{j \in N_d} y_{0j}^{k,f}), f \in F, k \in K_d \tag{16a}$$

$$\varepsilon_i^{k,f} + \varepsilon_{ij}^{k,f} + \tilde{\varepsilon}_j^{k,f} + \bar{\varepsilon}_j^{k,f} - B(1 - y_{ij}^{k,f}) \le \varepsilon_j^{k,f}$$
$$\le \varepsilon_i^{k,f} + \varepsilon_{ij}^{k,f} + \tilde{\varepsilon}_j^{k,f} + \bar{\varepsilon}_j^{k,f} + B(1 - y_{ij}^{k,f}), i \in \{0\} \cup N_d, j \in N_d, f \in F, k \in K_d \tag{16b}$$

$$\varepsilon_i^{k,f} + \varepsilon_{i,n+1}^{k,f} - B(1 - y_{i,n+1}^{k,f}) \le E, i \in N_d, f \in F, k \in K_d \tag{16c}$$

Constraints (13a) and (13b) calculate the energy consumed by each drone traversing each arc. Constraints (14a) and (14b) linearise the hover time a drone waits at each node in each trip. Constraints (15a) and (15b) calculate the energy consumed by each drone serving a customer, including the hover energy (if any). The Constraint (16a) means that each drone is equipped with a fully charged battery when leaving the depot. Constraint (16b) calculates the energy having consumed by a drone when leaving each customer location in each trip, and Constraint (16c) is used to avoid premature depletion of battery power when a drone returns to the depot. Note that Constraint (15b) is bilinear.

3.3.5. Load capacity constraints of trucks and drones

$$\sum_{i \in N_o} \sum_{j \in N_{do}} x_{ij}^k \cdot d_j = w_0^k \le Q, k \in K_t \tag{17a}$$

$$w_i^k - d_j - B(1 - x_{ij}^k) \le w_j^k \le w_i^k - d_j + B(1 - x_{ij}^k), i \in N_o, j \in N_c, k \in K_t \tag{17b}$$

$$Z_j^{k,f} \ge \sum_{i \in \{0\} \cup N_d} y_{i,j}^{k,f} - \sum_{i \in \{0\} \cup N_d} y_{i,j-P}^{k,f}, j \in N_d \cap N_{dc}, f \in F, k \in K_d \tag{18a}$$

$$Z_j^{k,f} \le 1 - \sum_{i \in \{0\} \cup N_d} y_{i,j-P}^{k,f}, Z_j^{k,f} \le \sum_{i \in \{0\} \cup N_d} y_{i,j}^{k,f}, j \in N_d \cap N_{dc}, f \in F, k \in K_d \tag{18b}$$

$$\sum_{i \in \{0\} \cup N_d} \sum_{j \in N_d \cap N_{do}} y_{ij}^{k,f} \cdot d_j + \sum_{j \in N_d \cap N_{dc}} Z_j^{k,f} \cdot d_j = \hat{w}_0^{k,f} \le \hat{Q}, f \in F, k \in K_d \tag{18c}$$

$$\hat{w}_i^{k,f} - d_j - B(1 - y_{ij}^{k,f}) \le \hat{w}_j^{k,f} \le \hat{w}_i^{k,f} - d_j + B(1 - y_{ij}^{k,f}), i \in \{0\} \cup N_d, j \in N_d, f \in F, k \in K_d \tag{18d}$$

Constraint (17a) gives a load capacity constraint on the total delivery-only quantities each truck can carry when leaving the depot. Constraint (17b) tracks the quantities loaded on each truck when leaving each customer location. Constraints (18a) and (18b) define that if a delivery-pair node j is served in trip f while its counterpart $j - P$ is not served in it, $Z_j^{k,f} = 1$, and 0 otherwise. Constraint (18c) imposes a load capacity constraint on the total quantities each drone can carry when leaving the depot. Constraint (18d) tracks the quantities loaded on a drone after serving each node.

In addition, to enhance the efficiency of the software solver in model solving, the following strengthening cuts are added to weed out infeasible routing, given the relatively limited capacity constraints of drones. They stipulate that variable $y_{ij}^{k,f}$ is fixed as 0 if the following conditions hold.

Lemma 1. *If* $2\lambda \cdot w_0 \cdot \frac{r_{0j}}{v} + \lambda \cdot w_0 \cdot s_j + \lambda |d_j| \frac{r_{0j}}{v} + \lambda \cdot max\{d_j, 0\} \cdot s_j > E$, *then* $\sum_{i \in \{0\} \cup N_d} y_{ij}^{k,f} \le 0, j \in N_{po} \cup N_{do}, f \in F, k \in K_d$.

Proposition 1. *When nodes i and j are paired, or not paired and* $d_i, d_j < 0$, *if* $\lambda(w_0 - d_i)\frac{\hat{r}_{ij}}{\hat{v}} \ge E$, *then* $y_{ij}^{k,f} \le 0$, $f \in F, k \in K_d, (i,j) \in A_d$.

Proposition 2. *For two unpaired nodes i and j, when* $d_j > 0$, *if* $\lambda(w_0 + d_j)\frac{\hat{r}_{ij}}{\hat{v}} \ge E$, *or* $\lambda(w_0 + d_j - d_i)\frac{\hat{r}_{ij}}{\hat{v}} \ge E$, $d_i < 0$, *then* $y_{ij}^{k,f} \le 0, f \in F, k \in K_d, (i,j) \in A_d$.

Proposition 3. *For two unpaired nodes i and j, when* $d_i < 0$ *or* $d_i, d_j > 0$, *if* $|d_i| + |d_j| > \hat{Q}$, *then* $y_{ij}^{k,f} \le 0, f \in F, k \in K_d, (i,j) \in A_d$.

The proofs of all Lemmas and Propositions in the paper can be found in Supplementary Materials A.

4. Hybrid metaheuristic

Given the NP-hard nature of the VRP, mPDPD-mT, as discussed above, is more complex, so solving its large-size problems in a reasonable time becomes rather challenging. Moreover, it is a new problem that cannot be directly addressed by existing solution approaches. As a consequence, we develop a custom metaheuristic for mPDPD-mT, referred to as improved large neighbourhood search (ILNS), which is based on the LNS framework. LNS was first proposed by Shaw (1997) and has shown good performance in various VRPs and strong applicability to new problems (Kitjacharoenchai et al., 2020; Wolfinger, 2021).

The ILNS consists of two sequential procedures for finding the best solutions. In particular, a construction heuristic (*Construction_heuristic*) is first developed to build an initial solution, which is then iteratively optimised via a large neighbourhood search embedded with random local searches (LNS-L) to obtain the best solution. In addition, to reduce drone hover energy, an effective algorithm is developed to determine drone waiting times at the depot. Algorithm 1 gives the framework.

Algorithm 1. *Outline of ILNS*

1: $S^{initial} \leftarrow Construction_heuristic()$
2: $S^{best} \leftarrow$ LNS-L $(S^{initial})$
3: **return** S^{best}

4.1. Initial solution construction

The *Construction_heuristic* procedure is initiated via a basic simulated annealing (SA) algorithm to build a truck-only TSP tour S_t that serves all pickup-only and delivery-only customers. Then, the obtained tour is split into multiple truck-only routes under the constraints in terms of truck load capacity and time windows. Third, *SeqConstTrip* is employed to designate some customers for drone services. Finally, each customer pair is relocated into the obtained parallel solution in sequence by *PairRelocation* for a complete solution. Note that all the procedures below are conducted only if the cost is reduced and the obtained solution is feasible. Algorithm 2 displays the framework of the *Construction_heuristic*. The computational complexity of this process is about $O(n^2)$.

Algorithm 2. *Construction_heuristic*

1: $S_t \leftarrow$ basic SA algorithm
2: $S_t \leftarrow$ split S_t if the current truck is full or violates the current time window
3: $S_{initial} \leftarrow SeqConstTrip(S_t)$
4: **for** $i \in N_{pc}$ **do**
5: $\quad S_{initial} \leftarrow PairRelocation\ (S_{initial}, i, i+P)$
6: **return** $S_{initial}$

SeqConstTrip. For each truck-only route $s \in S_t$, a list U_d for all drone-eligible customers is first identified; then, an attempt is made to construct a feasible trip for each customer in U_d one by one until one has been built or U_d has been traversed. Then, every other customer in U_d (if any) is inserted into the last executed trip (if any) to achieve cost reduction. Otherwise, a new feasible trip is started after the current drone flies back to the depot. Otherwise, a new drone is deployed.

PairRelocation. Each customer pair i and $i+P$ is iteratively assigned to the same truck or drone with the minimum cost increase, defined by the cost difference after and before relocation. A pickup pair i can be relocated to an existing truck route or an existing/new trip on an existing drone route or a new truck/drone route. However, for its counterpart, the relocation is more complex, as it can be relocated after node i on the same truck route, the same/another existing trip or a new trip on the same drone route, including the newly built truck/drone route. The possible relocations are displayed in Supplementary materials B.

Considering the complexity of the searches, the following acceleration strategies are applied to determine possible relocations and also used in follow-up algorithms. For a node ξ to be relocated to a truck route or a drone trip, denoted by an ordered set $s = \{0, i_1, \dots, i_j, \dots, i_{|s|-2}, n+1\}$, let φ_1^s and φ_2^s, initialised by $\varphi_1^s = 1$ and $\varphi_2^s = |s|-1$, be the lower and upper bound positions for relocating node ξ in s. If node ξ is a delivery pair and its counterpart is served by s at position b, $\varphi_1^s = b+1$. While the same case for node ξ which is a pickup pair, $\varphi_2^s = b$. Then we have

- if $l_{i_{|s|-2}} \le e_\xi$ and $\varphi_2^s = |s|-1$, $\varphi_1^s = |s|-1$;
- if $e_{i_1} \ge l_\xi$ and $\varphi_1^s = 1$, $\varphi_2^s = 1$;
- if $l_{i_j} \le e_\xi < l_{i_{j+1}}$, $\varphi_1^s = j+1$;
- if $e_{i_j} \ge l_\xi$, $\varphi_2^s = j$.

Moreover, the rules below are used to avoid infeasible trips for relocation. Let $\bigcup_{f\in\{1,\dots,F\}} s_f$, $s_f = \{0, i_1^f, \dots, i_j^f, \dots, i_{J_f}^f, n+1\}$, be the set of trips provided by a drone. For a trip s_f, let $[e_0^{s_f}, l_0^{s_f}]$ be the time window at its starting node (the depot 0) and $[e_{n+1}^{s_f}, l_{n+1}^{s_f}]$

be that at its ending node (the depot $n+1$). Then we have $e_0^{s_f} = e_{i_{J_{f-1}}^{f-1}}$ (when $f \geq 1$) and $l_0^{s_f} = l_{i_1^f}$; $e_{n+1}^{s_f} = e_{i_{J_f}^f}$ and $l_{n+1}^{s_f} = l_{i_1^{f+1}}$ (when $f \leq F-1$). Note that the e_0^1 of the first trip and l_{n+1}^F of the last trip are 0 and l_{max}, respectively. Moreover, let $\zeta_1^{s_f} = \sum_{i_j^f \in N_{do} \cap s_f} d_{i_j^f}$ and $\zeta_2^{s_f} = \sum_{i_j^f \in N_{po} \cap s_f} d_{i_j^f}$ be the total delivery-only and pickup-only quantities of trip s_f. Then,

- if $l_\xi \leq e_0^{s_f}$ or $e_\xi \geq l_{i_{n+1}}^{s_f}$, node ξ cannot be relocated to trip s_f;
- if $d_\xi > 0$ and $d_\xi + \zeta_1^{s_f} > \hat{Q}$, or $d_\xi < 0$ and $|d_\xi + \zeta_2^{s_f}| > \hat{Q}$, node ξ cannot be relocated to trip s_f.

4.2. Large neighbourhood search with random local searches (LNS-L)

LNS-L is implemented with a set of removal operators to destroy the current solution and a greedy repair procedure embedded with random local searches iteratively to obtain a better solution. For each iteration, the removal operators are adaptively selected according to their performance (weight, Γ). The weight of removal operator r after iteration j is updated by $\Gamma_{r,j+1} = \Xi * \Gamma_{r,j} + (1-\Xi) * \psi_r/\eta_r$, where ψ_r and η_r are score and use times, and $\Xi \in [0,1]$ is the reaction factor. The $\psi = \{\psi_1, \psi_2, \psi_3, \psi_4\}, \psi_1 > \psi_2 > \psi_3 > \psi_4$, corresponding to four performances of the new solution, which is the best, only improves current solution, poor but accepted and rejected, respectively. In addition, if a new solution is inferior to current solution, we accept it by SA principle. The overall scheme of the LNS-L is outlined in Algorithm 3 and explained later.

Algorithm 3. *LNS-L*

Input: $S_{initial}$, T_0, β, $iterMax$, T_f
Output: the best solution S_{best}

```
Initialisation: S_best ← S_initial, S_cur ← S_initial, T ← T_0, iter ← 0
S_initial ← Local_search(S_initial)
while T > T_f do
    if cost(S_cur) < cost(S_best) then
        S_best ← S_cur
    while iter < iterMax do
        S_new ← Destroy&Repair(S_cur)
        S_new ← Local_search(S_new)
        if cost(S_new) < cost(S_cur) then
            S_cur ← S_new
            if cost(S_new) < cost(S_best) then
                S_best ← S_new, update the score of destroy operators
            else
                Update the score of destroy operators
        else
            if U(0,1) < exp(cost(S_cur) − cost(S_new)/T) then
                S_cur ← S_new, update the score of destroy operators
            else
                Update the score of destroy operators
        iter+ = 1, update weight
    T ← T × β, iter ← 0
S^best ← Local_search(S^best)
return S_best
```

* T_0: initial temperature, β: cool rate, $iterMax$: number of iterations for each temperature, T_f: floor temperature.

4.2.1. Destroy operator

Six operators are screened to partially destroy the current solution. Note that if pickup or delivery pair customers are involved, their counterparts need to be removed as well. The computational complexity of this process is mainly determined by the last operator and is calculated as $T_D = O(d \cdot n' \cdot O(w))$, where d is the number of removed nodes, which is positively correlated with n. In each deletion, n' denotes the number of affected nodes and $O(w)$ denotes the computational complexity for cost calculation, incorporating waiting time decision for all affected trips.

- **Random removal** randomly deletes $max\{min\{n_1 - 1, 2\}, \lceil 10\% * n_1 \rceil\}$ truck-served non-pair nodes, $max\{min\{n_2 - 1, 3\}, \lceil 10\% * n_2 \rceil\}$ drone-served non-pair nodes and $max\{1, \lceil 40\% * P \rceil\}$ pair nodes, where $n_1 \geq 2$ and $n_2 \geq 2$ denote the numbers of non-pair customers served by trucks and drones, respectively.

- **Sweep removal** removes a set of customers adjacent in polar angles. It first randomly selects an angle ϕ from $[0, 359]$, then removes all nodes whose polar angles are in the range $[\phi_1, \phi_2]$ (when $\phi_2 > \phi_1$) or $[0, \phi_2] \cup [\phi_1, 360]$ (when $\phi_2 < \phi_1$). The ϕ_1 and ϕ_2 are determined by the formulae (19a) and (19b), where δ is generated from $max\{60 * U(0,1), 20\} + U(0,1)$.

$$\phi_1 = \phi - \delta + 360 * max\{-(\phi - \delta)/|\phi - \delta|, 0\} \tag{19a}$$

$$\phi_2 = \phi + \delta - 360 * max\{(\phi + \delta - 360)/|\phi + \delta - 360|, 0\} \tag{19b}$$

- **Cluster removal** ($n \geq 3$) targets at removing a cluster of nodes adjacent in Euclidean distances. Specifically, a seed is first randomly selected, and then the seed and the first $max\{min\{n-2, 2\}, \lceil 10\% * n \rceil\}$ nodes closest to it are removed in succession.
- **Similar removal** first randomly selects a seed, and then removes the seed along with $max\{min\{n - 2, 2\}, \lceil 10\% * n \rceil\}$ nodes that are most similar (with the smallest similarity coefficient SC) to it. The SC for two nodes i and j is defined as $\Phi_1(|e_i - e_j| + |l_i - l_j|) + \Phi_2|d_i - d_j| + \Phi_3|r_{ij} + \hat{r}_{ij}| + \Phi_4 * \chi_{ij}$, $\Phi_1, \Phi_2, \Phi_3, \Phi_4 \in [0, 1]$, where $\chi_{ij} \in \{0, 1\}$, and $\chi_{ij} = 1$ if these two nodes are both served by a truck or drone and 0 otherwise. Preliminary experiment determines the values of $(\Phi_1, \Phi_2, \Phi_3, \Phi_4)$ as (0.27,0.29,0.38,0.24).
- **Adjacent string removal** considers deleting two adjacent strings from different routes. Specifically, a seed is first randomly selected from a truck route or drone trip, and then the node closest to it in Euclidean distance served by another vehicle becomes the other seed. Finally, we randomly remove a string of length $max\{\lceil |s| * 30\% \rceil, 3\}$ around each seed (in route s) from current solution.
- **Worst removal** aims to iteratively delete a node or pair that reduces the largest cost Δ_C/n_3 after deletion until $max\{min\{n - 1, 3\}, \lceil 10\% * n \rceil\}$ nodes are deleted. Δ_C denotes cost difference before and after deleting the node or pair, and n_3 is the number of the deleted nodes in this deletion.

4.2.2. Repair operator

In this section, we introduce greedy relocation (GR) to repair the destroyed solution. Specifically, each deleted node i_c randomly selected from the set U_{rem} is repositioned to the best position with the minimum cost increase Δ_{min} consecutively until a complete solution is created. For each i_c, the GR evaluates all the possible relocating positions, including an existing truck route, an existing/new trip on an existing drone route and a new truck/drone route. Notably, if $i_c \in N_{pc} \cup N_{dc}$, GR becomes more challenging, as various relocations need to be considered, as shown in detail in Algorithm 4. The computational complexity of this process is approximately $T_R = O(d \cdot l \cdot O(w))$, where l denotes the number of possible insertion positions in the current solution.

Algorithm 4. *Greedy_relocation*

```
while U_rem ≠ ∅ do
    i_c ← randomly select a node from U_rem
    if i_c ∉ N_pc ∪ N_dc then
        if i_c ∉ N_d then
            S_new ← greedily relocate i_c into an existing/new truck route
        else
            S_new ← greedily relocate i_c into an existing or a new truck/drone route
        U_rem ← U_rem \ {i_c}
    else# the pickup pair and its counterpart are i_i^p and i_i^d, respectively
        S_new, Δ_min ← greedily relocate i_c^p and i_c^d into an existing/new truck route
        if i_c ∈ N_d then
            S', Δ_s' ← greedily insert i_c^p and i_c^d into one or two existing trips of a drone
            if Δ_s' < Δ_min then
                Δ_min ← Δ_s', S_new ← S'
            S', Δ_s' ← greedily insert i_c^p into an existing trip first and then construct a new trip for i_c^d after i_c^p
            if Δ_s' < Δ_min then
                Δ_min ← Δ_s', S_new ← S'
            S', Δ_s' ← greedily construct a new trip for i_c^p first and then insert i_c^d into an existing trip after i_c^p
            if Δ_s' < Δ_min then
                Δ_min ← Δ_s', S_new ← S'
            S', Δ_s' ← greedily construct one or two new trips for i_c^p and i_c^d in an existing route
            if Δ_s' < Δ_min then
                Δ_min ← Δ_s', S_new ← S'
            S', Δ_s' ← greedily start a new drone route for i_c^p and i_c^d (in one or two trips)
            if Δ_s' < Δ_min then
                Δ_min ← Δ_s', S_new ← S'
```

27: $U_{rem} \leftarrow U_{rem} \setminus \{i_c^p, i_c^d\}$

28: **return** S_{new}

4.2.3. Random local search

To increase diversity and reduce the risk of premature convergence to local optima, $Local_search$ is deployed by repeatedly running a group of random local moves to shake the solution and complement the repair procedure for further improvement. $Local_search$ includes inter-route and intra-route searches that work on different routes and within one route, respectively. The 'inter' and then 'intra' searches are invoked to produce a new solution in each iteration. We present the pseudocode of $Local_search$ in Algorithm 5. The computational complexity for a single iteration is approximately $T_L = O(n_{ter} \cdot O(w) + n_{tra} \cdot O(w))$, where n_{ter} and n_{tra} demote the numbers of operators in the inter- and intra-route local searches, respectively.

Algorithm 5. $Local_search$

Input: S-a feasible solution

Output: S'-optimised solution

1: Initialisation: $S' \leftarrow S$

2: **for** $\{1, ..., \omega\}$ **do** #repeat ω times

3: $S' \leftarrow$ 'inter' local search (S')

4: $S' \leftarrow$ 'intra' local search (S')

5: **return** S'

Inter-route local search

The 'inter' local search starts each of its operators sequentially producing a new feasible solution, with the best solution being a candidate in one iteration. Note that if paired nodes are involved, their counterparts are randomly relocated accordingly. Moreover, if a node cannot be inserted into existing drone trips, we consider greedily constructing a trip on a randomly selected route.

- **Truck node insertion** randomly relocates a pickup/delivery-only or two paired drone-eligible customers served by trucks to drone trips.
- **Drone node insertion** randomly relocates a pickup/delivery-only or two paired customers from drone trip(s) to a truck route.
- **General exchange** randomly exchanges two drone-eligible customers served by a truck route and a drone trip respectively.
- **Different truck exchange** conducts a random exchange between two customers from different truck routes.
- **Different drone exchange** conducts a random exchange between two customers from two trips by different drones.
- **Truck node move** randomly moves a pickup/delivery-only or two paired customers from a truck route to another one.
- **Drone node move** randomly reassigns a pickup/delivery-only or two paired customers from a drone trip to another one by other drones.
- **Trip construction** greedily creates one or two new trips for a pickup/delivery-only or two paired drone-eligible customers served by trucks on a randomly-selected route.

Intra-route local search

The following operators in 'intra' local search are performed for each truck and drone in sequence to produce neighbouring solutions, from which the best-feasible solution is selected. Note that for each exchange operator, if paired nodes are involved, their counterparts may also need to be randomly relocated accordingly.

- **Intra truck exchange** randomly exchanges two nodes served within a truck route.
- **Intra truck move** randomly moves a node to another position within a truck route. If a pickup (or delivery) pair node is selected, it must be inserted before (or after) its counterpart.
- **Intra drone exchange** randomly exchanges two nodes from the same or different trips of a drone.
- **Intra drone move** randomly moves a drone-served node to another position in the same or another trip within the same route. If infeasible after all attempts, a new trip is greedily constructed for it.

4.3. Waiting time optimisation for drones

As mentioned above, drones can wait at the depot before starting trips. Obviously, drone waiting times play a key role in hover energy consumption reduction and solution feasibility. Moreover, it is worth mentioning that waiting time optimisation must be deployed in every move at each iteration, which heavily increases computational time inevitably. To this end, we develop an effective algorithm with the following observations to tackle this problem for each trip.

To begin with, let τ_s^{ready} be the ready time for drone launch in a trip $s = \{i_0, i_1, i_2, \dots, i_j, \dots, i_J, n+1\}$, where J denotes the number of nodes served in it. That is, if trip s is the first trip of the drone, $\tau_s^{ready} = 0$. Otherwise, $\tau_s^{ready} = \hat{\tau}_{n+1}^{s-1} + S_r$. Note that $s_{i_0} = 0$,

and $e_i + s_i + \frac{\hat{r}_{ij}}{\hat{v}} \le l_j$ is valid for any arc (i, j) in the trip (otherwise, the trip is infeasible). For simplicity, we omit the index k of the same notations as in the model here. Then if $J \ge 2$, the waiting time of trip s at the depot is bounded by

$$\theta^s \ge max\{max\{e_{i_1}, min\{e_{i_2} - s_{i_1} - \frac{\hat{r}_{i_1,i_2}}{\hat{v}}, l_{i_1}\}\} - \tau_s^{ready} - S_l - s_{i_0} - \frac{\hat{r}_{i_0,i_1}}{\hat{v}}, 0\} \tag{20a}$$

$$\theta^s \le min\{l_{i_j} - \sum_{p\in\{0,\dots,j-1\}} s_{i_p} - \sum_{p\in\{0,\dots,j-1\}} \frac{\hat{r}_{i_p,i_{p+1}}}{\hat{v}}\} - \tau_s^{ready} - S_l, j \in \{1, \dots, J\} \tag{20b}$$

Proposition 4. *When $\tau_s^{ready} + S_l + \sum_{p\in\{0,\dots,j\}} s_{i_p} + \sum_{p\in\{0,\dots,j\}} \frac{\hat{r}_{i_p,i_{p+1}}}{\hat{v}} \ge e_{i_{j+1}}$, $j \in \{0, 1, \dots, J-1\}$, if $\theta^s = 0$, the hover energy consumption in trip s is zero, allowing the earliest ready time for subsequent service.*

Then we can deprive the Lemmas below based on different cases.

Lemma 2. *When $\tau_s^{ready} + S_l + s_{i_0} + \frac{\hat{r}_{i_0,i_1}}{\hat{v}} < e_{i_1}$ and when $J \ge 2$, $\tau_s^{ready} + S_l + \sum_{p\in\{0,\dots,j\}} s_{i_p} + \sum_{p\in\{0,\dots,j\}} \frac{\hat{r}_{i_p,i_{p+1}}}{\hat{v}} \ge e_{i_{j+1}}$, $j \in \{1, \dots, J-1\}$, if $\theta^s = e_{i_1} - S_l - \tau_s^{ready} - s_{i_0} - \frac{\hat{r}_{i_0,i_1}}{\hat{v}}$, the hover energy consumption in trip s is zero, allowing the earliest ready time for subsequent service.*

Lemma 3. *When $\tau_s^{ready} + S_l + s_{i_0} + \frac{\hat{r}_{i_0,i_1}}{\hat{v}} \ge e_{i_1}$, $\tau_s^{ready} + S_l + s_{i_0} + \frac{\hat{r}_{i_0,i_1}}{\hat{v}} + s_{i_1} + \frac{\hat{r}_{i_1,i_2}}{\hat{v}} < e_{i_2}$ and when $J \ge 3$, $\tau_s^{ready} + S_l + \sum_{p\in\{0,\dots,j\}} s_{i_p} + \sum_{p\in\{0,\dots,j\}} \frac{\hat{r}_{i_p,i_{p+1}}}{\hat{v}} \ge e_{i_{j+1}}$, $j \in \{2, \dots, J-1\}$, if $\theta^s = min\{e_{i_2} - s_{i_1} - \frac{\hat{r}_{i_1,i_2}}{\hat{v}}, l_i\} - \tau_s^{ready} - S_l - s_{i_0} - \frac{\hat{r}_{i_0,i_1}}{\hat{v}}$, the hover energy consumption in trip s is minimised, allowing the earliest ready time for subsequent service.*

Moreover, the following Propositions hold when $J = 2$ (Proposition 5) and $J = 3$ (Proposition 6).

Proposition 5. *For $J = 2$, when $\tau_s^{ready} + S_l + s_{i_0} + \frac{\hat{r}_{i_0,i_1}}{\hat{v}} < e_{i_1}$ and $\tau_s^{ready} + S_l + s_{i_0} + \frac{\hat{r}_{i_0,i_1}}{\hat{v}} + s_{i_1} + \frac{\hat{r}_{i_1,i_2}}{\hat{v}} < e_{i_2}$, the below conditions ensure that the hover energy consumption in trip s is minimised, allowing the earliest ready time for subsequent service. If $e_{i_1} + s_{i_1} + \frac{\hat{r}_{i_1,i_2}}{\hat{v}} > e_{i_2}$, $\theta^s = e_{i_1} - S_l - \tau_s^{ready} - s_{i_0} - \frac{\hat{r}_{i_0,i_1}}{\hat{v}}$. Otherwise, if $e_{i_2} - s_{i_1} - \frac{\hat{r}_{i_1,i_2}}{\hat{v}} \le l_{i_1}$, $\theta^s = max\{e_{i_2} - s_{i_1} - \frac{\hat{r}_{i_1,i_2}}{\hat{v}}, e_{i_1}\} - \tau_s^{ready} - S_l - s_{i_0} - \frac{\hat{r}_{i_0,i_1}}{\hat{v}}$; else, $\theta^s = l_{i_1} - \tau_s^{ready} - S_l - s_{i_0} - \frac{\hat{r}_{i_0,i_1}}{\hat{v}}$.*

Proposition 6. *For $J = 3$, when $\tau_s^{ready} + S_l + s_{i_0} + \frac{\hat{r}_{i_0,i_1}}{\hat{v}} \ge e_{i_1}$, $\tau_s^{ready} + S_l + s_{i_0} + \frac{\hat{r}_{i_0,i_1}}{\hat{v}} + s_{i_1} + \frac{\hat{r}_{i_1,i_2}}{\hat{v}} \ge e_{i_2}$ and $\tau_s^{ready} + S_l + s_{i_0} + \frac{\hat{r}_{i_0,i_1}}{\hat{v}} + s_{i_1} + \frac{\hat{r}_{i_1,i_2}}{\hat{v}} + s_{i_2} + \frac{\hat{r}_{i_2,i_3}}{\hat{v}} < e_{i_3}$, the below conditions ensure that the hover energy consumption in trip s is minimised, allowing the earliest ready time for subsequent service. If $e_{i_3} - s_{i_1} - \frac{\hat{r}_{i_1,i_2}}{\hat{v}} - s_{i_2} - \frac{\hat{r}_{i_2,i_3}}{\hat{v}} \le l_{i_1}$ and $e_{i_3} - s_{i_2} - \frac{\hat{r}_{i_2,i_3}}{\hat{v}} \le l_{i_2}$, $\theta^s = e_{i_3} - \tau_s^{ready} - S_l - s_{i_0} - \frac{\hat{r}_{i_0,i_1}}{\hat{v}} - s_{i_1} - \frac{\hat{r}_{i_1,i_2}}{\hat{v}} - s_{i_2} - \frac{\hat{r}_{i_2,i_3}}{\hat{v}}$. Otherwise, i.e., $e_{i_3} - s_{i_1} - \frac{\hat{r}_{i_1,i_2}}{\hat{v}} - s_{i_2} - \frac{\hat{r}_{i_2,i_3}}{\hat{v}} > l_{i_1}$ and $e_{i_3} - s_{i_2} - \frac{\hat{r}_{i_2,i_3}}{\hat{v}} \le l_{i_2}$, $\theta^s = l_{i_1} - \tau_s^{ready} - S_l - s_{i_0} - \frac{\hat{r}_{i_0,i_1}}{\hat{v}}$. Otherwise, i.e., $e_{i_3} - s_{i_1} - \frac{\hat{r}_{i_1,i_2}}{\hat{v}} - s_{i_2} - \frac{\hat{r}_{i_2,i_3}}{\hat{v}} \le l_{i_1}$ and $e_{i_3} - s_{i_2} - \frac{\hat{r}_{i_2,i_3}}{\hat{v}} > l_{i_2}$, $\theta^s = l_{i_2} - \tau_s^{ready} - S_l - s_{i_0} - \frac{\hat{r}_{i_0,i_1}}{\hat{v}} - s_{i_1} - \frac{\hat{r}_{i_1,i_2}}{\hat{v}}$. Otherwise, $\theta^s = min\{l_{i_1}, l_{i_2} - s_{i_1} - \frac{\hat{r}_{i_1,i_2}}{\hat{v}}\} - \tau_s^{ready} - S_l - s_{i_0} - \frac{\hat{r}_{i_0,i_1}}{\hat{v}}$.*

For all other cases, we find θ^s from the range defined by (20a) and (20b) by the following Algorithm 6. We first calculate the possible range $[\theta^{s,l}, \theta^{s,u}]$ of θ^s (line 1); then, gradually increase the value of θ^s at a rate of 0.1 from $\theta^{s,l}$ to $\theta^{s,u}$, and calculate corresponding total hover energy consumption E_s^h of trip s (lines 2–3). If $E_s^h = 0$, the corresponding θ^s is returned (lines 4–5). Otherwise, we return the θ^s with the minimum E_s^h (line 6).

Algorithm 6. *Waiting_time_optimisation* (when $J \ge 3$)

Input: τ_s^{ready}, the ready time for drone launch
Output: the best waiting time θ^s

1: $[\theta^{s,l}, \theta^{s,u}] \longleftarrow$ Eq. (20*a*) and (20*b*)
2: **for** $\theta^s \in [\theta^{s,l}, \theta^{s,u}]$ **do**
3: $\quad E_s^h \longleftarrow$ calculate the total hover energy consumption in s
4: $\quad$ **if** $E_s^h = 0$ **then**
5: $\quad\quad$ **return** θ^s
6: **return** the smallest θ^s with the minimum E_s^h

Finally, Algorithm 7 summaries the waiting time optimisation of trip s for all considered cases.

Algorithm 7. *Waiting time optimisation of trip s for all considered cases*

Input: τ_s^{ready}, the ready time for drone launch
Output: the best waiting time θ^s

Table 3
The timing flow for each best drone route of running example A.1.16.

Drone route	Trip s	r_s^{ready} (min)	Best waiting time (min)	Arrival time at the depot (min)
Route 1	trip 1: 0 → 10 → 17	0.00	54.35	66.65
	trip 2: 0 → 16 → 17	67.65	31.80	121.55
	trip 3: 0 → 15 → 12 → 17	122.55	42.81	186.38
Route 2	trip 1: 0 → 14 → 1 → 17	0.00	38.67	77.71
	trip 2: 0 → 2 → 4 → 17	78.71	0.00	114.77
	trip 3: 0 → 9 → 11 → 17	115.77	0.00	146.24
	trip 4: 0 → 3 → 7 → 17	147.24	6.89	191.89
	trip 5: 0 → 6 → 17	192.89	16.95	233.16

if $\tau_s^{ready} + S_l + \sum_{p\in\{0,\dots,j\}} s_{i_p} + \sum_{p\in\{0,\dots,j\}} \frac{\hat{r}_{i_p,i_{p+1}}}{\hat{v}} \geq e_{i_{j+1}}$, $j \in \{0,1,\dots,J-1\}$ **then**
  return $\theta^s \leftarrow$ Proposition 4
else
  if $\tau_s^{ready} + S_l + s_{i_0} + \frac{\hat{r}_{i_0,i_1}}{\hat{v}} < e_{i_1}$ and when $J \geq 2$, $\tau_s^{ready} + S_l + \sum_{p\in\{0,\dots,j\}} s_{i_p} + \sum_{p\in\{0,\dots,j\}} \frac{\hat{r}_{i_p,i_{p+1}}}{\hat{v}} \geq e_{i_{j+1}}$, $j \in \{1,\dots,J-1\}$ **then**
    return $\theta^s \leftarrow$ Lemma 2
  else
    if $\tau_s^{ready} + S_l + s_{i_0} + \frac{\hat{r}_{i_0,i_1}}{\hat{v}} \geq e_{i_1}$, $\tau_s^{ready} + S_l + s_{i_0} + \frac{\hat{r}_{i_0,i_1}}{\hat{v}} + s_{i_1} + \frac{\hat{r}_{i_1,i_2}}{\hat{v}} < e_{i_2}$ and when $J \geq 3$, $\tau_s^{ready} + S_l + \sum_{p\in\{0,\dots,j\}} s_{i_p} + \sum_{p\in\{0,\dots,j\}} \frac{\hat{r}_{i_p,i_{p+1}}}{\hat{v}} \geq e_{i_{j+1}}$, $j \in \{2,\dots,J-1\}$ **then**
      return $\theta^s \leftarrow$ Lemma 3
    else
      if $J = 2$ **then**
        return $\theta^s \leftarrow$ Proposition 5
      else
        if $J = 3$ and $\tau_s^{ready} + S_l + s_{i_0} + \frac{\hat{r}_{i_0,i_1}}{\hat{v}} \geq e_{i_1}$, $\tau_s^{ready} + S_l + s_{i_0} + \frac{\hat{r}_{i_0,i_1}}{\hat{v}} + s_{i_1} + \frac{\hat{r}_{i_1,i_2}}{\hat{v}} \geq e_{i_2}$, $\tau_s^{ready} + S_l + s_{i_0} + \frac{\hat{r}_{i_0,i_1}}{\hat{v}} + s_{i_1} + \frac{\hat{r}_{i_1,i_2}}{\hat{v}} + s_{i_2} + \frac{\hat{r}_{i_2,i_3}}{\hat{v}} < e_{i_3}$ **then**
          return $\theta^s \leftarrow$ Proposition 6
        else
          return $\theta^s \leftarrow$ Algorithm 6

Remark 1 (*Execution of the Best Drone Waiting Times*)**.** For any given trip s of a drone with ready time (r_s^{ready}) input, the best waiting time of the drone at the depot before starting the trip s can be determined via Algorithm 7. Recall that for each drone route r_0^{ready} for the first trip is zero, and once the best waiting time for a trip s is determined, r_{s+1}^{ready} for the next trip can be easily calculated (arrival time at the depot + retrieval time, referred to as "r_{s+1}^{ready} calculation"). Therefore, for a given drone route, all best waiting times for all trips can be obtained by repeatedly conducting the "Algorithm 7–r_{s+1}^{ready} calculation" process. For a running example (A.1.16), the best drone routes are performed by two drones in 3 and 5 trips, respectively. Then the time flow for each drone route can be obtained as in Table 3.

Remark 2 (*Complexity of ILNS*)**.** The computational complexity of LNS_L for a single iteration is about $T_D + T_R + \omega \cdot T_L = O(d \cdot n' \cdot O(w) + d \cdot l \cdot O(w) + \omega \cdot (n_{ter} \cdot O(w) + n_{tra} \cdot O(w)))$. Then the computational complexity of ILNS is calculated as $O(n^2 + n_{iter}(d \cdot n' \cdot O(w) + d \cdot l \cdot O(w) + \omega \cdot (n_{ter} \cdot O(w) + n_{tra} \cdot O(w)))$, where n_{iter} is the number of iterations. In fact, the complexity of the ILNS is greatly increased because the problem concerned is significantly more complicated than the general PDSVRP, especially regarding pair services, multiple drone visits and trips, and hover energy and waiting time decisions due to time windows, which will be verified by subsequent experiments. We have managed to develop efficient algorithms to find near-optimal solutions to the routing and timing of trucks and drones in parallel, but complexity may impact the size of solvable problems at the expense of more computational time if the problems scale up massively.

5. Numerical experiments

In this part, various numerical experiments are carried out to study the validity of the developed MIBLP/strengthening inequalities, the effectiveness of the proposed ILNS, and the superiority of key problem features. All generated test instances and the required values for all the parameters are introduced in Section 5.1. Then, Section 5.2 introduces the solutions provided for all considered instances and extensively evaluates the performance of proposed ILNS algorithm. In Section 5.3, we investigate the effectiveness of drone waiting time optimisation at the depot. In Section 5.4, we conduct two comparative experiments to discuss the key features of our problem. Finally, a sensitivity analysis of four key parameters is carried out in Section 5.5. The developed MIBLP was solved with Gurobi 11.0.2, and the proposed ILNS and experiments were implemented in Python 3.11.7 and run on an Intel(R) Core(TM) i7-10750H @ 2.60 GHz processor with 32 GB of memory.

Table 4
Summary of parameters.

(a) Model parameters.

Parameter	Value	Reference
Weight of a no-load drone (w_0)	6 kg	Meng et al. (2023)
Unit energy consumption of drones (λ)	2/3 Wh/(kg min)	–
Load capacity of trucks (Q)	750 kg	Meng et al. (2024)
Load capacity of drones ($\hat{Q}$)	3 kg	Liu et al. (2020)
Battery capacity of drones (E)	180 Wh ($=\eta \cdot (\hat{Q} + w_0) \cdot T$), where $T = 0.5$ h is the battery endurance	Murray and Chu (2015)
Constant speed of trucks (v)	30 km/h	Zhen et al. (2023)
Constant speed of drones ($\hat{v}$)	40 km/h	Nguyen et al. (2022)
Travel cost per unit distance of trucks (c)	\$0.78/km	Salama and Srinivas (2020)
Energy cost per unit energy of drones ($\hat{c}$)	\$0.0104 ($=\frac{c}{\varsigma} \cdot \hat{v} \cdot T/E$), where $\varsigma = 8.3$ is the unit cost ratio of truck to drone (abbreviated as vehicle cost ratio)	Salama and Srinivas (2020)
Times spent launching and retrieving drone (S_l, S_r)	1 min, 1 min	Murray and Chu (2015)
Time spent serving node i (s_i)	2 min	Tong et al. (2022)
Fixed cost for deploying a truck (c^0)	\$15	–
Fixed cost for deploying a drone ($\hat{c}^0$)	\$5	–

(b) Metaheuristic parameters.

Descriptions	Values	Descriptions	Values
Parameters used in the ILNS		***Scores used in the ILNS***	
T_0- initial temperature	50	ψ_1- score of a new best solution	7
β- cooling rate	0.90	ψ_2- score of a new improving solution	3
$iterMax$- number of iterations	10	ψ_3- score of an accepted poor solution	1
Ξ- reaction factor	0.7	ψ_4- score of a rejected solution	0
ω- number of iterations in local searches	40		

5.1. Instance sets and parameters

Given that no benchmark instances consider similar models, we choose to randomly generate instances for all the experiments. First, we evenly generate six sets of test instances, each with 12 instances and a size of $n = \{30, 50, 70, 100, 150, 200\}$. All node locations for each instance are uniformly distributed, as described in Liu et al. (2020), on a plane with two different areas: 15×15 (for $n = \{30, 50\}$) and 20×20 (for $n = \{70, 100, 150, 200\}$) (unit: km^2). We note that as long as drones can serve a certain number of customers from the depot, the proposed model and methodology are not tied to any particular location distributions (as confirmed in Section 5.5.2). Each set contains three time window scenarios (four for each); namely, for each customer i, we randomly generate (A) $e_i \in [20, 260]$ and $l_i = e_i + 20$, (B) $e_i \in [60, 360]$ and $l_i = e_i + 20$, and (C) $e_i \in [20, 260]$ and $l_i = e_i + 60$. The time window for the depot is set to [0,450]. In each instance, we set the first 10% and 30% of the customers as pair nodes and pickup-only nodes, respectively, and the remaining customers as delivery-only nodes. We randomly sample the demands of 90% of the customers within the range of (0, 2.3], and the demands of the remaining 10% are generated in the range of (3, 50] (Zhen et al., 2023). In addition, we randomly create 20 small instances with 14 or 16 customers (evenly divided), on the basis of scenario A, to validate the correctness of the MIBLP and the performance of the ILNS and strengthening inequalities. For convenience, we name each instance 'A/B/C.#.n'. All instances can be obtained from https://doi.org/10.17632/hnxgm8c6zr.4.

Table 4a and b summarise the values of most model parameters and parameters used in ILNS, respectively.

5.2. Experiments and results

5.2.1. Strengthening inequalities and model evaluation with small-size instances

In this section, the best results obtained by Gurobi and the ILNS for the 20 small instances are compared to assess the correctness of the MIBLP and the effectiveness of the strengthening inequalities and the developed ILNS. Table 5 presents the detailed comparative results, which include the best solver solutions without and with strengthening inequalities (C^{G_1} and C^{G_2}), their corresponding run times ($time^{G_1}$ and $time^{G_2}$), the best solution among 5 runs obtained by the ILNS (C) and their average computational time ($\overline{time}$). Note that the run time of Gurobi for each instance is limited to 2 h, and the numbers of trucks and drones available are set to 3 and 5, respectively. The percentage gaps between them are defined by $\Delta_1\%(= 100(C^{G_2} - C^{G_1})/C^{G_1})$ and $\Delta_2\%(= 100(C - C^{G_2})/C^{G_2})$, respectively. Note that since our designed algorithm is not limited to the types of time windows, we select scenario A as an example. The detailed results of the small-size instances in scenarios B and C can be found in Supplementary Materials C.

In terms of computational time, we note that Gurobi consumes less computational time when the strengthening inequalities are included, and the run time is clearly reduced by more than 20% on average, highlighting the effectiveness of these inequalities. Moreover, the run time of Gurobi increases substantially when more customers are served and fluctuates heavily even for the same-size instances, whereas that of the ILNS is much smaller and more robust.

With respect to solution quality, for all considered instances with 14 customers, both Gurobi and the ILNS can provide optimal solutions, but the ILNS consumes much less run time on average. When 16 customers are served, the ILNS can still provide the same

Table 5
Solution comparison between ILNS and Gurobi for small instances.

Instance	Solver solution				ILNS		Δ_1%	Δ_2%	Instance	Solver solution				ILNS		Δ_1%	Δ_2%
	C^{G_1} (\$)	$time^{G_1}$ (s)	C^{G_2} (\$)	$time^{G_2}$ (s)	C (\$)	$\overline{time}$ (s)				C^{G_1} (\$)	$time^{G_1}$ (s)	C^{G_2} (\$)	$time^{G_2}$ (s)	C (\$)	$\overline{time}$ (s)		
A.1.14	52.33	158.37	52.33	121.03	52.33	97.89	0.00	0.00	A.1.16	57.56	292.32	57.56	210.31	57.56	108.03	0.00	0.00
A.2.14	46.91	127.32	46.91	95.15	46.91	115.04	0.00	0.00	A.2.16	48.40	2414.24	48.40	1985.96	48.40	89.15	0.00	0.00
A.3.14	45.90	153.63	45.90	126.89	45.90	101.59	0.00	0.00	A.3.16	61.01	2677.63	61.01	1896.36	61.01	108.53	0.00	0.00
A.4.14	41.13	223.35	41.13	189.36	41.13	117.69	0.00	0.00	A.4.16	59.40	893.53	59.40	693.05	59.40	150.01	0.00	0.00
A.5.14	48.28	190.92	48.28	159.36	48.28	126.43	0.00	0.00	A.5.16	**48.33**	**7200.00**[a]	**48.32**	**5983.26**	48.33	115.44	−0.02	0.02
A.6.14	47.77	396.27	47.77	319.53	47.77	179.12	0.00	0.00	A.6.16	56.03	3232.10	56.03	2253.26	56.03	286.68	0.00	0.00
A.7.14	47.83	1454.14	47.83	957.63	47.83	96.62	0.00	0.00	A.7.16	**58.26**	**7200.00**[a]	**58.16**	**6742.82**	58.16	145.17	−0.17	0.00
A.8.14	53.76	646.17	53.76	359.75	53.76	17.18	0.00	0.00	A.8.16	51.51	**7200.00**[a]	51.51	**5843.09**	51.51	205.10	0.00	0.00
A.9.14	49.83	5051.93	49.83	4520.36	49.83	121.05	0.00	0.00	A.9.16	59.00	1870.46	59.00	1176.97	59.00	127.87	0.00	0.00
A.10.14	50.98	3169.32	50.98	1983.69	50.98	84.68	0.00	0.00	A.10.16	49.24	711.08	49.24	581.63	49.24	211.62	0.00	0.00
Avg.		**1157.14**		**883.28**		**105.73**	**0.000**	**0.000**	**Avg.**		**3307.93**		**2636.67**		**154.76**	**−0.019**	**0.002**

[a] Gurobi cannot obtain an optimal solution within 2 h.

or near-optimal solution (one minor gap) and requires much shorter time than Gurobi does, demonstrating the good performance of the proposed ILNS. Moreover, the strengthening inequalities are effective, as all three instances that cannot be solved optimally by Gurobi within 2 h are exactly solved, and two of them obtain better solutions when these strengthening inequalities are considered (marked in bold).

5.2.2. Algorithm performance on medium and large instances

In this part, further analyses are conducted on the results of the mPDPD-mT using the 72 medium and large instances created in Section 5.1. Tables 6 and 7 display the best result among the 5 runs solved by the ILNS for each. Each result includes the total costs of the truck-only mode and the mPDPD-mT (C_t and C), truck travel distances (R'_t and R_t), and the number of vehicles deployed (U'_t and U_t, U_d). Moreover, they also display the hover cost (C_h) and the detailed characteristics of each best solution, including the numbers of trips (F) and customers assigned to drones (R). The ratios of customers assigned to drones to drone-eligible customers (D) and the hover cost to the total cost are defined in columns $\Delta_{r/d}(= 100 * R/D)$ and $\Delta_{h/c}(= 100 * C_h/C)$, respectively. Then, columns $\Delta_{R_t,R'_t}(= 100(R_t - R'_t)/R'_t)$ and $\Delta_{C,C_t}(= 100(C - C_t)/C_t)$ calculate the percentage gaps in the distance travelled by trucks and total cost in comparison to the truck-only mode, respectively. Finally, the average computational time is recorded in column $\overline{time}$.

In general, the scenario with wider time windows (C) has a lower total cost and requires fewer trucks and drones than the scenarios with tighter time windows (A and B). For the solution characteristics, over 90% and 75% of drone-eligible customers are assigned to drones on average in instances with distribution areas of 15 × 15 and 20 × 20, respectively, and the percentages are relatively stable over different instances of the same size. Moreover, the solutions in scenario C can provide almost a similar number of drone services but with fewer drone trips than those in the other two scenarios. In all scenarios, the hover energy cost is relatively stable, accounting for a small portion of the total cost over different instance sizes (≤0.48%), and even almost zero in scenario C, demonstrating that considering drone waiting times is effective. A more detailed evaluation is given in Section 5.3, where we can observe that if drones are not allowed to wait at the depot, the total cost could be much higher.

In addition, although larger instances typically take longer computational times, 46 out of 72 instances can be solved within 1 h, and the longest computational time is still less than 2 h. Moreover, compared with other scenarios, the longest computational time is required in the instances of scenario C owing to their widest time windows.

Furthermore, compared with the case of only deploying trucks, the mPDPD-mT can save over 40% of the cost on average and requires fewer trucks; therefore, the distance travelled by trucks decreases more than 60% on average, demonstrating its significant cost advantage over the truck-only mode.

Finally, to further explore the performance of the ILNS, we compare it with the large neighbourhood search (LNS) designed by Kitjacharoenchai et al. (2020), the slack induction by string and sweep removals (SISSRs) developed by Nguyen et al. (2022), and the adaptive large neighbourhood search (ALNS) provided by Jiang et al. (2024) on all instances with 30 and 50 customers. LNS is designed for multi-visit VRPD, SISSRs are used to solve the single-visit PDSVRP, and ALNS is deployed for pickup and delivery VRPD, which share some similarities with the mPDPD-mT. Moreover, to adapt to our problem, some problem-tailored procedures in ILNS, such as initial solution construction, acceleration strategies and waiting time optimisation of drones, are also employed in these three benchmarks. The total numbers of iterations for LNS, SISSRs and ALNS are limited to 1500, 800 and 3500, respectively. The average and best total costs among 5 runs and the average computational times are reported in Table 8, along with their cost percentage gaps, which are calculated by $\Delta_{l,i}\% = 100(C_l - C)/C$; $\Delta_{s,i}\% = 100(C_s - C)/C$; $\Delta_{a,i}\% = 100(C_a - C)/C$.

In general, the ILNS outperforms the three alternatives in terms of the quality of the solutions, especially for larger-size instances. The gaps of LNS, SISSRs and ALNS from ILNS are approximately 5% on average. In addition, the ILNS is robust over problems of different sizes, followed by ALNS, SISSRs, and then LNS, in which the gaps between the average and the best results (Δ%) are the largest. Specifically, LNS and ALNS are more effective than SISSRs for smaller instances (with 30 customers). LNS can quickly deliver a better quality solution than SISSRs with less computational time owing to its simple destroy operators and greedy repair and acceptance criteria of solutions. ALNS extends LNS by using an adaptive selection mechanism and more effective operators (such as similarity removal), thereby obtaining a better and more robust solution than LNS does. However, operators such as the worst removal might slightly increase the computational time. Although SISSRs are effective for the single-visit parallel mode, they may not be very applicable to multi-visit cases with pickup and time windows. For example, the local searches are dedicated only

Table 6
Solutions produced by ILNS for medium instances.

Instance	C_t ($)	R'_t (km)	U'_t	C ($)	C_h ($)	R_t (km)	U_t	U_d	F	R	D	$\Delta_{r/d}$%	$\Delta_{h/c}$%	Δ_{R_t,R'_t}%	Δ_{C,C_t}%	$\overline{time}$ (s)
A.1.30	184.93	179.40	3	72.78	0.01	30.60	1	3	15	26	27	96.30	0.01	−82.94	−60.64	681.62
A.2.30	159.66	147.00	3	74.89	0.07	46.80	1	2	12	24	27	88.89	0.09	−68.16	−53.09	770.23
A.3.30	173.86	165.21	3	73.43	0.00	40.80	1	2	13	22	27	81.48	0.00	−75.30	−57.76	581.72
A.4.30	165.28	154.21	3	68.59	0.03	37.60	1	2	10	24	27	88.89	0.04	−75.62	−58.50	1112.25
B.1.30	165.43	154.40	3	60.20	0.14	25.60	1	2	13	26	27	96.30	0.23	−83.42	−63.61	903.49
B.2.30	160.75	148.40	3	66.80	0.00	44.80	1	3	13	26	27	96.30	0.00	−69.81	−58.44	1170.29
B.3.30	175.57	167.40	3	75.61	0.29	44.20	1	2	13	24	27	88.89	0.38	−73.60	−56.93	1134.07
B.4.30	178.85	171.60	3	69.09	0.20	28.60	1	3	13	26	27	96.30	0.29	−83.33	−61.37	840.86
C.1.30	142.32	144.00	2	62.87	0.04	27.60	1	2	12	26	27	96.30	0.06	−80.83	−55.82	1609.31
C.2.30	131.24	129.79	2	68.71	0.00	48.80	1	1	10	20	27	74.07	0.00	−62.40	−47.65	1028.69
C.3.30	135.92	135.79	2	70.28	0.12	36.40	1	2	13	26	27	96.30	0.17	−73.19	−48.29	1299.82
C.4.30	134.21	133.60	2	58.85	0.00	36.40	1	1	7	21	27	77.78	0.00	−72.75	−56.15	1622.03
Avg.												**89.81**	**0.11**	**−75.11**	**−56.52**	
A.1.50	228.79	216.40	4	110.18	0.53	52.80	2	3	18	40	45	88.89	0.48	−75.60	−51.84	1213.77
A.2.50	231.41	219.76	4	111.18	0.09	55.80	2	3	19	39	45	86.67	0.08	−74.61	−51.96	1085.36
A.3.50	221.30	206.79	4	98.51	0.13	35.80	2	3	18	42	45	93.33	0.13	−82.69	−55.49	987.59
A.4.50	226.30	213.21	4	95.39	0.26	52.80	1	3	20	42	45	93.33	0.27	−75.24	−57.85	1804.71
B.1.50	221.15	206.60	4	100.41	0.15	59.00	1	3	18	42	45	93.33	0.15	−71.44	−54.60	1662.81
B.2.50	244.08	236.00	4	101.15	0.35	57.40	1	3	19	41	45	91.11	0.35	−75.68	−58.56	1932.34
B.3.50	257.81	253.60	4	111.47	0.15	57.20	2	3	19	40	45	88.89	0.13	−77.45	−56.76	1168.23
B.4.50	253.44	248.00	4	108.70	0.23	68.20	1	3	20	42	45	93.33	0.21	−72.50	−57.11	854.26
C.1.50	183.37	177.40	3	81.77	0.00	47.00	1	2	15	41	45	91.11	0.00	−73.51	−55.41	1767.58
C.2.50	191.80	188.21	3	84.51	0.00	50.40	1	2	14	38	45	84.44	0.00	−73.22	−55.94	2752.06
C.3.50	179.83	172.86	3	80.02	0.00	41.40	1	2	15	41	45	91.11	0.00	−76.05	−55.50	1320.82
C.4.50	172.76	163.79	3	80.80	0.05	45.80	1	2	15	39	45	86.67	0.06	−72.04	−53.23	2332.85
Avg.												**90.18**	**0.16**	**−75.00**	**−55.35**	
A.1.70	387.96	382.00	6	224.45	0.03	159.60	3	4	23	44	58	75.86	0.01	−58.22	−42.15	1472.11
A.2.70	382.68	356.00	7	207.43	0.12	120.00	3	6	28	50	59	84.75	0.06	−66.29	−45.80	1392.49
A.3.70	368.15	356.60	6	216.22	0.18	137.00	3	5	27	49	58	84.48	0.08	−61.58	−41.27	2420.71
A.4.70	380.63	372.60	6	185.83	0.02	117.20	2	5	26	53	59	89.83	0.01	−68.55	−51.18	1552.33
B.1.70	392.46	407.00	5	175.97	0.11	93.80	2	5	31	55	61	90.16	0.06	−76.95	−55.16	2290.70
B.2.70	375.63	366.19	6	186.04	0.36	94.80	2	6	33	55	61	90.16	0.19	−74.11	−50.47	1257.16
B.3.70	370.82	340.79	7	186.63	0.17	93.00	3	5	28	55	62	88.71	0.09	−72.71	−49.67	1949.42
B.4.70	394.20	390.00	6	233.15	0.00	157.80	3	5	26	46	58	79.31	0.00	−59.54	−40.85	1447.83
C.1.70	305.54	314.79	4	185.55	0.14	131.60	2	4	21	48	58	82.76	0.08	−58.19	−39.27	4155.87
C.2.70	301.51	290.40	5	168.19	0.00	107.40	2	4	23	49	59	83.05	0.00	−63.02	−44.22	3120.95
C.3.70	289.63	294.40	4	184.50	0.02	136.80	2	4	17	42	58	72.41	0.01	−53.53	−36.30	3499.68
C.4.70	285.58	289.21	4	170.11	0.00	119.20	2	3	20	49	59	83.05	0.00	−58.78	−40.43	3496.42
Avg.												**83.71**	**0.05**	**−64.29**	**−44.73**	

to exploring inter-route neighbourhood structures. Furthermore, owing to the complexity of our problem as discussed before, such greedy local searches performed in every iteration inevitably lead to more time being spent. However, when the size of instances increases (to 50 customers), SISSRs outperform them, although they maintain a 5% gap from the ILNS. The local searches are time consuming, but in turn, they may help to provide slightly better results. In contrast, the performances of the LNS and ALNS degrade rapidly. Especially for LNS, the results obtained fluctuate greatly over different instances due to the lack of operators that target the problem characteristics and the risk of premature convergence to local optima. Although ALNS improves LNS, thereby significantly enhancing the robustness of the solution, its effectiveness is still weakened because of the lack of consideration of problem features such as time windows and insufficiently comprehensive searches. In terms of run time, ILNS is more efficient than the three alternatives are, especially for larger-size instances, which require the least computational time on average. ILNS has the best performance because of its customised design, which targets problem characteristics such as time windows and pair services. Notably, ILNS customises multiple destroy operators and adopts an adaptive mechanism, increasing the exploration and pertinence of selection. Moreover, combining random local search with greedy repair improves both search diversity and repair quality.

5.2.3. Comparison with the PDSVRP benchmark

To further evaluate the performance of the ILNS, we compare it with the SISSRs developed by Nguyen et al. (2022) for solving the PDSVRP. The comparative results are presented in Table 9, in which the best results of the single-visit PDSVRP (C_{sv}) and multi-visit PDSVRP (C_{mv}) using ILNS are both provided. We note that for the SISSRs, the average result of the initial solutions ($\overline{z_{init}}$), the result of the best-found solutions (z_{init}) and the average computational time ($\overline{time}$) are in line with those in Nguyen et al. (2022). We refer readers to Nguyen et al. (2022) for a detailed description of the PDSVRP instances and parameter setting.

Overall, ILNS can provide competitive results in similar computational times (within 20 s) for most test instances compared with those of SISSRs, demonstrating the effectiveness of the proposed ILNS in solving the PDSVRP. Regarding the quality of the initial solution, we note that the ILNS can provide a better solution (marked in bold). In addition, drone parameters appear to significantly affect the benefits of multiple drone visits, as expected, allowing multiple drone visits certainly reduces the total cost,

Table 7
Solutions produced by ILNS for large instances.

Instance	C_t (\$)	R'_t (km)	U'_t	C(\$)	C_h (\$)	R_t (km)	U_t	U_d	F	R	D	$\Delta_{r/d}$%	$\Delta_{h/c}$%	Δ_{R_t,R'_t}%	Δ_{C,C_t}%	$\overline{time}$ (s)
A.1.100	506.12	475.79	9	324.10	0.04	227.00	5	6	29	57	83	68.67	0.01	−52.29	−35.96	2758.57
A.2.100	548.69	549.60	8	300.16	0.20	204.20	4	6	32	66	81	81.48	0.07	−62.85	−45.30	3236.94
A.3.100	461.33	437.60	8	212.03	0.24	80.40	3	8	42	84	90	93.33	0.11	−81.63	−54.04	4852.85
A.4.100	498.32	465.79	9	306.36	0.07	200.40	4	7	34	67	80	83.75	0.02	−56.98	−38.52	3468.56
B.1.100	499.55	486.60	8	256.96	0.06	154.00	3	6	41	70	88	79.55	0.02	−68.35	−48.56	3264.57
B.2.100	475.84	456.21	8	305.58	0.37	197.40	4	7	38	61	82	74.39	0.12	−56.73	−35.78	2308.73
B.3.100	492.68	477.79	8	257.22	0.00	168.40	3	6	35	68	85	80.00	0.00	−64.75	−47.79	4257.16
B.4.100	475.99	456.40	8	300.70	0.84	210.20	4	5	33	59	83	71.08	0.28	−53.94	−36.83	3554.66
C.1.100	407.02	387.21	7	269.15	0.00	196.20	4	4	23	54	83	65.06	0.00	−49.33	−33.87	4564.12
C.2.100	407.93	407.60	6	244.89	0.00	173.00	3	5	25	60	81	74.07	0.00	−57.56	−39.97	5163.87
C.3.100	350.96	353.79	5	172.85	0.03	73.00	2	6	36	82	90	91.11	0.02	−79.37	−50.75	6052.96
C.4.100	402.65	381.60	7	246.52	0.00	168.00	3	5	30	64	80	80.00	0.00	−55.97	−38.78	5056.23
Avg.												**78.54**	**0.05**	**−61.65**	**−42.18**	
A.1.150	640.98	591.00	12	408.59	0.00	282.00	6	7	44	93	126	73.81	0.00	−52.28	−36.26	3889.46
A.2.150	667.48	644.21	11	400.11	0.11	276.20	5	9	44	92	120	76.67	0.03	−57.13	−40.06	3309.34
A.3.150	608.80	588.21	10	289.78	0.08	123.40	3	12	57	125	135	92.59	0.03	−79.02	−52.40	4847.44
A.4.150	655.33	609.40	12	394.86	0.00	283.00	4	8	48	98	124	79.03	0.00	−53.56	−39.75	1815.60
B.1.150	641.58	611.00	11	379.53	0.25	256.60	5	7	47	92	129	71.32	0.07	−58.00	−40.84	3419.86
B.2.150	627.07	592.40	11	378.77	0.00	233.20	5	9	52	95	126	75.40	0.00	−60.63	−39.60	4357.26
B.3.150	622.68	606.00	10	375.58	0.75	250.60	5	7	48	94	127	74.02	0.20	−58.65	−39.68	3830.46
B.4.150	613.32	594.00	10	391.52	0.00	266.00	5	8	46	92	127	72.44	0.00	−55.22	−36.16	2640.86
C.1.150	511.90	483.21	9	319.58	0.22	212.00	6	6	37	99	126	78.57	0.07	−56.13	−37.57	6983.28
C.2.150	508.26	517.00	7	323.62	0.08	229.00	5	7	36	93	120	77.50	0.02	−55.71	−36.33	4806.91
C.3.150	484.42	467.21	8	246.91	0.00	109.40	3	8	47	118	135	87.41	0.00	−76.58	−49.03	5984.24
C.4.150	492.37	477.40	8	318.47	0.00	219.20	4	7	40	99	124	79.84	0.00	−54.08	−35.32	6330.16
Avg.												**78.22**	**0.04**	**−59.75**	**−40.25**	
A.1.200	810.76	770.21	14	504.94	0.01	353.60	7	9	51	112	167	67.07	0.00	−54.09	−37.72	3499.26
A.2.200	845.44	776.21	16	544.90	0.15	398.40	8	11	65	122	162	75.31	0.03	−48.67	−35.55	4893.10
A.3.200	768.79	716.40	14	372.37	0.27	153.80	4	15	74	162	180	90.00	0.07	−78.53	−51.56	6517.06
A.4.200	796.72	752.21	14	487.98	0.25	325.80	5	11	66	126	162	77.78	0.05	−56.69	−38.75	5556.67
B.1.200	815.77	757.40	15	515.66	0.28	341.20	8	8	58	134	170	78.82	0.05	−54.95	−36.79	3834.86
B.2.200	795.00	750.00	14	473.37	0.25	315.40	6	9	59	120	164	73.17	0.05	−57.95	−40.46	4331.37
B.3.200	793.02	709.00	16	461.07	0.41	301.40	6	9	61	122	172	70.93	0.09	−57.49	−41.86	3974.15
B.4.200	778.28	747.79	13	448.95	0.67	279.80	6	11	66	125	162	77.16	0.15	−62.58	−42.32	5358.75
C.1.200	666.28	604.21	13	428.93	0.00	296.00	6	7	47	117	167	70.06	0.00	−51.01	−35.62	5690.92
C.2.200	667.50	625.00	12	443.50	0.00	323.20	7	6	37	109	162	67.28	0.00	−48.29	−33.56	6921.31
C.3.200	586.96	560.21	10	326.49	0.00	149.20	4	10	64	155	180	86.11	0.00	−73.37	−44.38	7168.12
C.4.200	617.87	580.60	11	392.14	0.00	227.80	5	8	54	129	162	79.63	0.00	−60.76	−36.53	4523.61
Avg.												**76.11**	**0.04**	**−58.70**	**−39.59**	

but the average reduction rate is relatively low (2.70%), since all test instances consider a single-visit scenario, providing rather limited drone capacity and endurance. However, allowing multiple drone visits comes at the expense of dramatically increasing the computing time (almost doubling) and significantly increases the complexity of the problem. In more detail, the cost savings appear to be inconsistent, with gaps broadly ranging from 0.24% to 12.02% across 27 instances, which may be due to different distributions of nodes and the depot; for a detailed analysis, see Section 5.5.2.

5.2.4. Comparison with CVRP optimal solutions

Inspired by Kitjacharoenchai et al. (2020), we compare the ILNS for solving the mPDPD-mT with the CVRP optimal solutions for further investigation. A total of 27 CVRP benchmark instances are considered for the experiment. Considering that the drone cost is a key factor in drone deployment, we set two levels of the vehicle cost ratio (ς) for comparison. For each test instance, the best result among 5 runs, average computational time and gaps over the CVRP optimal solution are recorded in Table 10. For the ILNS, we set the costs of trucks and drones per unit distance to 1 and ς, respectively. To adapt to the CVRP benchmark instances, we relax the time window constraint and assume a fixed range of drones in the mPDPD-mT.

The results show that the gaps between the ILNS and the optimal CVRP solution are relatively consistent over different cost levels, and the computational times are similar (the smaller ones appear in bold), indicating that the ILNS is effective and reliable for solving the parallel mode. In addition, the ILNS provides a lower cost by 3.72% if $\varsigma = 2$ and by 12.45% if $\varsigma = 2.5$ on average. That is, if the vehicle cost ratio is less than 2, the advantages of the parallel model are not particularly significant. Otherwise, such as $\varsigma = 2.5$, the benefits of the parallel mode increase rapidly. We conclude that the advantages of the parallel mode are prominent when the vehicle cost ratio is above 2. Given the low-cost nature of drones, implementing the parallel mode is still advantageous, especially when time constraints are considered. A detailed sensitivity analysis of the drone cost in parallel mode is presented in Section 5.5.3.

Table 8
ILNS vs. LNS, SISSRs and ALNS for solving the mPDPD-mT.

Instance	ILNS				LNS					SISSRs					ALNS				
	C (\$)	$\overline{C}$ (\$)	Δ%	$\overline{time}$ (s)	C_l(\$)	$\overline{C}$(\$)	Δ%	$\overline{time}$ (s)	$\Delta_{l,i}$%	C_s (\$)	$\overline{C}$(\$)	Δ%	$\overline{time}$ (s)	$\Delta_{s,i}$%	C_a(\$)	$\overline{C}$ (\$)	Δ%	$\overline{time}$ (s)	$\Delta_{a,i}$%
A.1.30.20	72.78	73.16	0.52	681.62	72.78	79.12	8.71	**523.45**	0.00	81.73	85.16	4.20	1178.19	12.30	73.66	74.06	0.54	536.26	1.21
A.2.30.20	74.89	77.06	2.90	770.23	75.20	81.56	8.46	**523.17**	0.41	75.20	80.46	6.99	1506.03	0.41	75.93	77.98	2.70	705.69	1.39
A.3.30.20	73.43	75.86	3.31	581.72	78.20	84.63	8.22	**427.13**	6.50	78.85	82.61	4.77	992.09	7.38	74.08	76.92	3.83	646.97	0.89
A.4.30.20	68.59	70.63	2.97	1112.25	72.18	76.59	6.11	**649.31**	5.23	68.72	71.29	3.74	1977.58	0.19	73.80	75.97	2.94	1135.52	7.60
B.1.30.20	60.20	62.96	4.58	903.49	60.28	71.25	18.20	**661.73**	0.13	66.11	70.51	6.66	1484.06	9.82	60.72	63.82	5.11	1284.04	0.86
B.2.30.20	66.80	67.69	1.33	1170.29	66.80	70.69	5.82	**506.27**	0.00	67.18	69.33	3.20	1130.24	0.57	67.41	69.54	3.16	872.32	0.91
B.3.30.20	75.61	76.05	0.58	1134.07	75.61	82.62	9.27	**507.26**	0.00	77.80	81.43	4.67	1178.96	2.90	75.82	76.13	0.41	791.31	0.28
B.4.30.20	69.09	70.46	1.98	840.86	69.09	76.16	10.23	**640.54**	0.00	69.71	72.06	3.37	1103.74	0.90	69.83	70.91	1.55	937.41	1.07
C.1.30.60	62.87	64.29	2.26	1609.31	62.87	67.61	7.54	**1439.71**	0.00	65.61	71.26	8.61	3203.09	4.36	63.26	65.62	3.73	2833.33	0.62
C.2.30.60	68.71	70.04	1.94	**1028.69**	72.53	81.60	12.51	1315.61	5.56	73.43	77.28	5.24	1778.25	6.87	68.71	71.76	4.44	1643.44	0.00
C.3.30.60	70.28	72.41	3.03	1299.82	70.29	78.03	11.01	**1185.20**	0.01	75.48	80.41	6.53	1945.73	7.40	70.65	73.91	4.61	1827.73	0.53
C.4.30.60	58.85	59.88	0.73	1622.03	61.61	67.97	10.32	**1493.95**	4.69	65.54	69.03	5.32	2431.15	11.37	58.85	60.09	2.11	1793.81	0.00
Avg.			**2.18**				**9.70**		**1.88**			**5.28**		**5.37**			**2.93**		**1.28**
A.1.50.20	110.18	114.03	3.49	**1213.77**	112.74	119.43	5.93	1638.36	2.32	113.75	115.94	1.93	1644.31	3.24	116.36	119.68	2.85	1257.12	5.61
A.2.50.20	111.18	113.24	1.85	**1085.36**	111.67	120.10	7.55	1910.21	0.44	115.45	115.63	0.16	1469.67	3.84	113.89	116.56	2.34	1200.38	2.44
A.3.50.20	98.51	100.36	1.88	**987.59**	103.13	111.53	8.15	1362.30	4.69	106.39	107.53	1.07	1545.49	8.00	102.80	106.30	3.40	1009.50	4.35
A.4.50.20	95.39	98.07	2.81	1804.71	103.24	109.34	5.91	1751.52	8.23	100.22	105.93	5.70	2077.84	5.06	104.46	108.65	4.01	**1466.37**	9.51
B.1.50.20	100.41	103.81	3.39	1662.81	105.30	112.29	6.64	**1424.27**	4.87	105.56	106.27	0.67	1972.10	5.13	107.95	109.12	1.08	1675.36	7.51
B.2.50.20	101.15	104.50	3.31	1932.34	101.60	108.41	6.70	**1841.61**	0.44	105.23	111.82	6.26	2015.79	4.03	105.09	108.50	3.24	1965.62	3.90
B.3.50.20	111.47	112.34	0.78	**1168.23**	115.43	124.40	7.77	1534.16	3.55	119.04	120.19	0.97	1226.86	6.79	114.88	116.01	0.98	1152.59	3.06
B.4.50.20	108.70	111.07	2.18	**854.26**	134.46	142.62	6.07	1826.47	23.70	112.05	116.19	3.69	1822.48	3.08	115.42	120.53	4.43	1275.12	6.18
C.1.50.60	81.77	82.29	0.64	**1767.58**	101.87	108.18	6.19	3160.89	24.58	87.43	90.08	3.03	3608.39	6.92	92.47	96.94	4.83	3432.66	13.09
C.2.50.60	84.51	87.06	3.02	2752.06	86.01	93.23	8.39	**2567.58**	1.77	89.66	91.30	1.83	3335.17	6.09	88.00	91.61	4.10	2722.28	4.13
C.3.50.60	80.02	83.93	4.89	**1320.82**	85.01	93.06	9.47	2392.07	6.24	87.74	92.43	5.35	2891.80	9.65	85.30	88.46	3.70	2545.51	6.60
C.4.50.60	80.80	83.82	3.74	**2332.85**	94.61	102.32	8.15	2730.08	17.09	87.14	93.26	7.02	2996.94	7.85	88.60	90.92	2.62	3763.59	9.65
Avg.			**2.67**				**7.24**		**8.16**			**3.14**		**5.81**			**3.13**		**6.34**

Table 9
ILNS vs. SISSRs for solving PDSVRP from Nguyen et al. (2022).

Instance	SISSRs			ILNS					$\Delta_{C_{sv},z_{best}}$	$\Delta_{C_{mv},z_{best}}$
	$\overline{z_{init}}$ (\$)	z_{best} (\$)	$\overline{time}$ (s)	$\overline{C_{init}}$ (\$)	C_{sv} (\$)	$\overline{time}$ (s)	C_{mv} (\$)	$\overline{time}$ (s)	%	%
50-r-0-c	287.95	161.13	15.35	**223.49**	161.13	14.54	150.22	34.11	0.00	−6.77
50-r-0-e	435.21	310.50	14.33	**391.39**	310.50	11.26	309.50	22.52	0.00	−0.32
50-r-0-r	353.81	246.53	13.79	**332.25**	246.53	16.94	244.69	26.15	0.00	−0.75
50-r-1-c	280.84	177.21	16.28	**230.10**	177.21	12.84	170.78	25.70	0.00	−3.63
50-r-1-e	370.02	216.97	14.59	**358.40**	216.97	17.32	215.04	36.59	0.00	−0.89
50-r-1-r	355.04	228.11	15.30	**327.83**	228.11	15.51	226.51	26.09	0.00	−0.70
50-r-2-c	278.73	181.49	13.56	**222.70**	181.49	13.49	176.41	28.08	0.00	−2.80
50-r-2-e	500.12	375.05	12.75	**443.14**	375.05	13.62	373.88	20.61	0.00	−0.31
50-r-2-r	380.61	313.32	12.90	**376.23**	313.32	13.94	311.64	19.60	0.00	−0.54
50-c-0-c	151.73	89.04	16.43	**103.02**	89.04	11.47	85.18	30.68	0.00	−4.34
50-c-0-e	255.69	182.91	15.53	**218.04**	182.91	14.61	179.74	23.89	0.00	−1.73
50-c-0-r	153.15	112.17	15.59	**123.69**	112.17	18.52	110.87	35.12	0.00	−1.16
50-c-1-c	116.83	81.76	17.12	**96.23**	81.76	15.97	78.61	26.56	0.00	−3.85
50-c-1-e	177.55	133.57	12.94	**146.58**	133.57	11.27	132.48	17.29	0.00	−0.82
50-c-1-r	196.76	148.86	15.92	**180.45**	148.86	12.99	146.91	30.93	0.00	−1.31
50-c-2-c	154.70	77.16	19.45	**131.53**	77.16	15.74	73.16	35.85	0.00	−5.18
50-c-2-e	292.78	214.25	16.97	**276.02**	214.25	13.19	212.14	21.77	0.00	−0.98
50-c-2-r	186.17	136.51	17.37	**159.01**	136.51	11.61	134.21	21.36	0.00	−1.68
50-rc-0-c	236.97	116.80	19.06	**163.06**	109.87	15.14	108.71	26.41	−5.93	−6.93
50-rc-0-e	345.61	242.02	14.64	**302.67**	242.02	11.48	241.43	23.08	0.00	−0.24
50-rc-0-r	295.49	217.87	15.63	**275.60**	217.87	16.94	216.25	27.01	0.00	−0.74
50-rc-1-c	227.81	110.44	20.72	**151.80**	110.44	15.27	97.17	34.64	0.00	−12.02
50-rc-1-e	324.39	201.62	15.74	**266.55**	201.62	16.73	200.18	25.46	0.00	−0.71
50-rc-1-r	299.11	173.16	16.69	**233.97**	173.16	18.10	171.90	32.40	0.00	−0.73
50-rc-2-c	229.04	129.65	18.14	**203.78**	129.82	14.94	114.12	30.12	0.13	−11.98
50-rc-2-e	487.22	383.49	12.95	**485.22**	383.49	11.14	381.95	12.90	0.00	−0.40
50-rc-2-r	272.75	186.31	15.72	**243.76**	186.31	16.57	183.86	26.31	0.00	−1.32
Avg.									**−0.21**	**−2.70**

Table 10
ILNS for solving the mPDPD-mT vs. CVRP optimal solutions.

Instance	n	$\lvert K_t \rvert$	Q	$\lvert K_d \rvert$	$\hat{Q}$	CVRP	ILNS ($\varsigma = 2.0$)			ILNS ($\varsigma = 2.5$)		
						(optimal)	C (\$)	$\overline{time}$ (s)	$\Delta_{i,opt}$%	C (\$)	$\overline{time}$ (s)	$\Delta_{i,opt}$%
A-n32-k5	31	5	100	2	35	784	773.32	19.85	−1.36	689.47	**13.16**	−12.06
A-n33-k5	32	5	100	2	35	661	624.87	19.90	−5.47	543.15	**16.47**	−17.83
A-n33-k6	32	6	100	2	35	742	709.54	**12.87**	−4.37	645.92	17.37	−12.95
A-n34-k5	33	5	100	2	35	778	714.91	24.05	−8.11	653.69	**9.88**	−15.98
A-n36-k5	35	5	100	2	35	799	765.29	**16.03**	−4.22	670.09	17.87	−16.13
A-n37-k5	36	5	100	2	35	669	602.26	26.72	−9.98	497.80	**16.23**	−25.59
A-n37-k6	36	6	100	2	35	949	928.92	14.56	−2.12	843.42	**11.93**	−11.13
A-n38-k5	37	5	100	2	35	730	681.76	17.54	−6.61	590.80	**15.46**	−19.07
A-n39-k5	38	5	100	2	35	822	791.17	**14.28**	−3.75	700.07	14.76	−14.83
A-n39-k6	38	6	100	2	35	831	772.37	32.42	−7.06	662.10	**18.80**	−20.32
A-n44-k6	43	6	100	2	35	937	931.20	**16.66**	−0.62	795.11	22.53	−15.14
A-n45-k6	44	6	100	2	35	944	915.96	**21.22**	−2.97	836.40	27.12	−11.40
A-n46-k7	45	7	100	2	35	914	880.80	30.99	−3.63	812.95	**27.09**	−11.06
A-n48-k7	47	7	100	2	35	1073	1049.40	**20.40**	−2.20	982.95	31.82	−8.39
A-n53-k7	52	7	100	2	35	1010	988.34	**18.97**	−2.14	887.63	20.54	−12.12
A-n54-k7	53	7	100	2	35	1167	1167.00	**18.58**	0.00	1069.18	27.83	−8.38
A-n55-k9	54	9	100	2	35	1073	1061.64	**18.72**	−1.06	989.06	25.28	−7.82
A-n62-k8	61	8	100	2	35	1288	1288.00	**21.12**	0.00	1215.16	33.41	−5.66
A-n63-k10	62	10	100	2	35	1314	1286.76	27.73	−2.07	1221.12	**25.19**	−7.07
A-n65-k9	64	9	100	2	35	1174	1161.39	**29.64**	−1.07	1076.16	37.04	−8.33
A-n69-k9	68	9	100	2	35	1159	1116.13	**21.67**	−3.70	1007.41	41.98	−13.08
B-n31-k5	30	5	100	2	35	672	645.96	24.32	−3.87	601.87	**7.59**	−10.44
B-n34-k5	33	5	100	2	35	788	750.63	10.63	−4.74	712.89	**6.41**	−9.53
B-n35-k5	34	5	100	2	35	955	867.87	22.59	−9.12	843.87	**5.96**	−11.64
B-n38-k6	37	6	100	2	35	805	755.63	28.98	−6.13	709.28	**12.14**	−11.89
B-n39-k5	38	5	100	2	35	549	527.91	13.64	−3.84	488.15	**10.36**	−11.08
B-n41-k6	40	6	100	2	35	829	827.78	**14.25**	−0.15	769.20	17.16	−7.21
Avg.									**−3.72**			**−12.45**

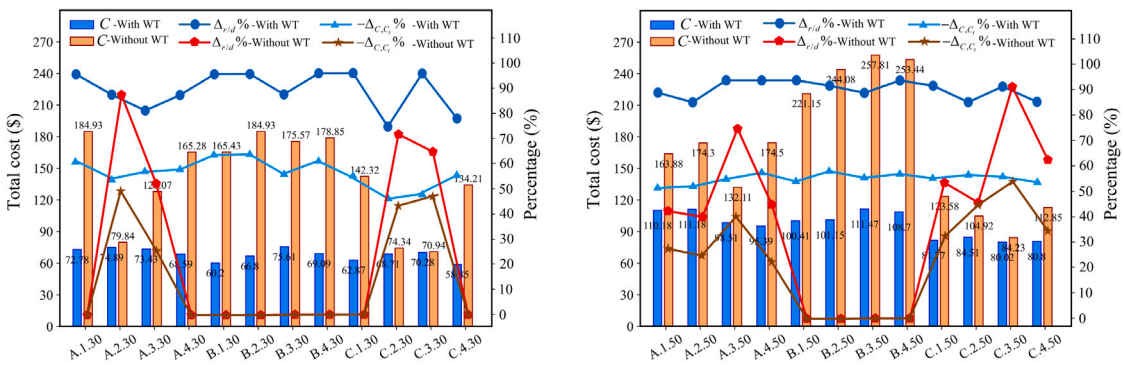

Fig. 4. Comparison of cases with and without WT.

5.3. Impact of waiting time optimisation of drones

To quantitatively evaluate the benefits of drone waiting time optimisation at the depot (WT), we compare the total cost, ratio of customers assigned to drones ($\Delta_{r/d}$%) and cost reduction over truck-only mode ($-\Delta_{C,C_t}$%) for cases with and without WT for instances with 30 and 50 customers, as shown in Fig. 4. This indicates that in the absence of waiting times, performance could be adversely affected, as the total cost is substantially increased compared with that of the case with WT. Moreover, the increase is more remarkable in scenario B than in scenarios A and C, and the number of drone-served customers has even been reduced to zero due to the later opening time windows, resulting in much greater waiting time at customer locations. Additionally, with waiting time optimisation, the two percentage values are relatively robust, especially in instances with more customers, and can be kept at levels of over 80% and 55%, respectively. However, in the case without WT, the percentage fluctuates greatly over different instances.

Table 11
Impact of allowing multiple drone visits.

Instance	Multiple visits							Single visit							Δ_C%	Δ_F%	Δ_R%	$\Delta_{\overline{time}}$%
	C ($)	R_t (km)	U_t	U_d	F	R	$\overline{time}$ (s)	C($)	R_t (km)	U_t	U_d	F	R	$\overline{time}$ (s)				
A.1.50	110.18	**52.80**	2	3	18	40	1213.77	127.23	58.80	2	4	36	36	218.71	15.47	100.00	−10.00	−81.98
A.2.50	111.18	55.80	2	3	19	39	1085.36	124.81	55.80	2	4	38	38	254.41	12.26	100.00	−2.56	−76.56
A.3.50	98.51	**35.80**	2	3	18	42	987.59	112.96	36.60	2	4	41	41	244.16	14.67	127.78	−2.38	−75.28
A.4.50	95.39	**52.80**	1	3	20	42	1804.71	110.50	53.60	1	4	41	41	236.03	15.84	105.00	−2.38	−86.92
B.1.50	100.41	**59.00**	1	3	18	42	1662.81	116.86	59.80	1	4	40	40	278.42	16.38	122.22	−4.76	−83.26
B.2.50	101.15	**57.40**	1	3	19	41	1932.34	115.15	59.60	1	4	39	39	268.29	13.84	105.26	−4.88	−86.12
B.3.50	111.47	57.20	2	3	19	40	1168.23	125.17	**56.80**	2	4	40	40	227.06	12.29	110.53	0.00	−80.56
B.4.50	108.70	68.20	1	3	20	42	854.26	122.67	**67.80**	1	4	42	42	238.46	12.85	110.00	0.00	−72.09
C.1.50	81.77	**47.00**	1	2	15	41	1767.58	105.74	48.20	1	4	38	38	148.69	29.31	153.33	−7.32	−91.59
C.2.50	84.51	50.40	1	2	14	38	2752.06	97.51	**47.20**	1	3	38	38	160.68	15.38	171.43	0.00	−94.16
C.3.50	80.02	**41.40**	1	2	15	41	1320.82	96.48	43.20	1	3	39	39	148.34	20.57	160.00	−4.88	−88.77
C.4.50	80.80	45.80	1	2	15	39	2332.85	97.46	45.80	1	3	39	39	194.08	20.62	160.00	0.00	−91.68
Avg.															**16.62**	**127.13**	**−3.26**	**−84.08**

5.4. Comparisons with different modes

To explore the key features of the problem more deeply and understand the results, we perform two additional experiments, namely, comparisons with the cases allowing only single visit or single trip of drones, on all A/B/C.#.50 instances.

5.4.1. Multiple visits vs. single visit

In this section, we aim to explore the benefits of allowing multiple drone visits in parallel operations compared with the case where all drones are allowed to make only single-visit trips. Table 11 summarises the best result among 5 runs for each considered instance. As can be observed, allowing multiple drone visits is advantageous, as the total cost is obviously reduced, although more computational time is consumed (in line with the results in Section 5.2.3). Furthermore, the number of trucks required is the same, and the truck travel distance still seems similar (the smaller ones appear in bold), but more drones are required to complete more trips (even doubling) to reduce cost, while a limited number of customers are served in the single-visit case.

In addition, a total of four key parameters, namely drone capacities (load $\hat{Q}$ and battery E), drone speed $\hat{v}$ and time window width wd are identified to have clear impacts on the performance of multiple drone visits. The best results of several selected instances among 5 runs are reported in Table 12, in which three levels are set for each parameter and R/F denotes the ratio of the number of customers served by the drones to the number of trips executed. In general, the larger the drone capacities, the faster the drone speed, or the wider the time windows, the greater the performance of multiple drone visits within reasonable limits. Specifically, drone battery capacity has a greater impact on multiple visits than drone load capacity does; however, even when one of the capacities drops to 50% of the baseline, allowing multiple visits is still effective compared with the single-visit case. Moreover, even if the drones slow down to the same speed as the trucks do (although drones are usually faster than trucks), allowing multiple visits is still effective, the ratio R/F can be kept at 1.73% on average. In addition, as long as the width of time windows is no less than 20 min, at least two customers can be served per trip for most instances, and the ratio R/F exceeds three on average if the width is increased to 90 min.

5.4.2. Multiple trips vs. single trip

This section focuses on exploring the benefits of allowing multiple trips by drones. We compare the mPDPD-mT (multi-trip mode) with the case that considers single drone trips (single-trip mode). Table 13 presents the best results among the 5 runs, with reported results similar to those of previous tables. Overall, the advantages of multiple trips are obvious, as the total cost and truck travel distance increase by approximately 65% and 50%, respectively, in single-trip mode, and the numbers of trips and drone-served customers decrease by 36.75% and 20.05%, respectively. Moreover, the single-trip mode requires more vehicles (marked in bold) and does not make full use of drones. Despite the apparent increase in the number of drones deployed, fewer drone trips are still initiated; consequently, fewer customers are served by drones. In addition, allowing multiple trips may increase the problem complexity as the CPU time in the mPDPD-mT is longer.

Moreover, drone capacities are identified as the key factors affecting the performance of multiple drone trips. The best results of the test instances among 5 runs are reported in Table 14, where F/U_d denotes the ratio of the number of trips executed to the number of drones used. Overall, the number of trips conducted per drone decreases as the drone capacities increase within reasonable limits. Specifically, allowing multiple drone trips is more affected when the drone capacities are relatively small (50% of the baseline). However, when the capacities reach or exceed the baseline, the changes are not obvious and the ratio F/U_d can still be maintained above 5 on average.

5.5. Sensitivity analysis

Considering that the performance of the parallel concept depends heavily on factors such as demand proportions, node distributions, drone cost and battery capacity, we deeply explore their impacts in this section. Note that we only vary the value of one considered parameter at a time, leaving the rest unvaried to monitor the effect of the parameter on the investigated results.

Table 12
Impact of key parameters on the performance of multiple drone visits.

Instance		$\hat{Q}$ (kg)			E (Wh)			$\hat{v}$ (km/h)			wd (min)		
		1.5	3.0	6.0	90	180	240	30	40	60	20	60	90
A.1.30	C($)	117.68	72.78	64.91	146.16	72.78	71.83	100.03	72.78	61.79	72.78	62.87	60.87
	R/F%	1.42	1.73	2.00	1.00	1.73	2.00	1.53	1.73	2.00	1.73	2.17	2.78
A.2.30	C($)	109.57	74.89	74.68	99.45	74.89	74.34	85.87	74.89	70.92	74.89	68.71	60.77
	R/F%	1.46	2.00	2.18	1.20	2.00	2.18	1.71	2.00	2.00	2.00	2.00	2.63
B.1.30	C($)	103.99	60.20	51.30	120.31	60.20	59.79	74.29	60.20	55.71	60.20	58.28	52.56
	R/F%	2.00	2.00	2.15	1.36	2.00	2.36	1.85	2.00	2.36	2.00	2.36	2.60
B.2.30	C($)	111.18	66.80	57.93	138.67	66.80	61.86	96.52	66.80	57.47	66.80	59.18	58.08
	R/F%	1.60	2.00	2.17	1.09	2.00	2.00	1.62	2.00	1.86	2.00	2.60	2.67
A.1.50	C($)	169.06	110.18	105.81	179.21	110.18	109.44	129.08	110.18	101.57	110.18	81.77	79.92
	R/F%	1.77	2.22	2.35	1.22	2.22	2.67	1.71	2.22	2.67	2.22	2.73	3.08
A.2.50	C($)	167.77	111.18	104.16	180.37	111.18	110.18	130.06	111.18	101.80	111.18	84.51	78.68
	R/F%	2.17	2.05	2.16	1.41	2.05	2.44	1.80	2.05	2.44	2.05	2.71	3.33
B.1.50	C($)	154.10	100.41	99.71	174.73	100.41	99.69	114.41	100.41	93.37	100.41	89.94	82.76
	R/F%	2.08	2.33	2.47	1.05	2.33	2.47	1.95	2.33	2.73	2.33	3.17	3.80
B.2.50	C($)	173.74	101.15	100.42	188.97	101.15	100.53	121.73	101.15	93.02	101.15	92.53	78.17
	R/F%	1.73	2.16	2.33	1.28	2.16	2.41	1.68	2.16	2.16	2.16	2.56	3.25
Avg.	C	**138.39**	**87.20**	**82.37**	**153.48**	**87.20**	**85.96**	**106.50**	**87.20**	**79.46**	**87.20**	**74.72**	**68.98**
	R/F	**1.78**	**2.06**	**2.23**	**1.20**	**2.06**	**2.32**	**1.73**	**2.06**	**2.23**	**2.06**	**2.54**	**3.02**

Table 13
Impact of allowing multiple drone trips.

Instance	Multiple trips							Single trip							Δ_C%	Δ_{R_t}%	Δ_F%	Δ_R%	$\Delta_{\overline{time}}$%
	C ($)	R_t (km)	U_t	U_d	F	R	$\overline{time}$ (s)	C ($)	R_t (km)	U_t	U_d	F	R	$\overline{time}$ (s)					
A.1.50	110.18	52.80	2	3	18	40	1213.77	168.79	67.20	2	**13**	13	37	814.98	53.19	27.27	−27.78	−7.50	−32.86
A.2.50	111.18	55.80	2	3	19	39	1085.36	172.36	89.00	2	**11**	11	33	761.74	55.03	59.50	−42.11	−15.38	−24.91
A.3.50	98.51	35.80	2	3	18	42	987.59	160.21	56.60	2	**13**	13	35	851.66	62.63	58.10	−27.78	−16.67	−22.87
A.4.50	95.39	52.80	1	3	20	42	1804.71	164.86	62.20	**2**	**13**	13	34	734.92	72.83	17.80	−35.00	−19.05	−52.81
B.1.50	100.41	59.00	1	3	18	42	1662.81	167.84	75.00	**2**	**12**	12	32	921.42	67.15	27.12	−33.33	−23.81	−44.59
B.2.50	101.15	57.40	1	3	19	41	1932.34	168.55	71.80	1	**15**	15	34	936.98	66.63	25.09	−21.05	−17.07	−51.51
B.3.50	111.47	57.20	2	3	19	40	1168.23	173.17	58.20	2	**15**	15	39	640.16	55.35	1.75	−21.05	−2.50	−45.20
B.4.50	108.70	68.20	1	3	20	42	854.26	176.30	157.20	**2**	**10**	10	30	804.92	62.19	130.50	−50.00	−28.57	−5.78
C.1.50	81.77	47.00	1	2	15	41	1767.58	138.71	62.20	**2**	**9**	9	32	1357.76	69.63	32.34	−40.00	−21.95	−23.19
C.2.50	84.51	50.40	1	2	14	38	2752.06	141.94	75.20	**2**	**8**	8	31	1895.97	67.96	49.21	−42.86	−18.42	−31.11
C.3.50	80.02	41.40	1	2	15	41	1320.82	135.81	84.00	**2**	**6**	6	24	2001.32	69.72	102.90	−60.00	−41.46	51.52
C.4.50	80.80	45.80	1	2	15	39	2332.85	144.99	71.60	**2**	**9**	9	28	1273.38	79.44	56.33	−40.00	−28.21	−45.42
Avg.															**65.15**	**48.99**	**−36.75**	**−20.05**	**−35.98**

Table 14
Impact of key parameters on the performance of multiple drone trips.

Instance		$\hat{Q}$ (kg)			E (Wh)			Instance		$\hat{Q}$ (kg)			E (Wh)		
		1.5	3.0	6.0	90	180	240			1.5	3.0	6.0	90	180	240
A.1.30	C($)	117.68	72.78	64.91	146.16	72.78	71.83	A.1.50	C($)	169.06	110.18	105.81	179.21	110.18	109.44
	F/U_d%	6.00	5.00	4.67	11.00	5.00	4.33		F/U_d%	6.50	6.00	5.67	9.00	6.00	5.00
A.2.30	C($)	109.57	74.89	74.68	99.45	74.89	74.34	A.2.50	C($)	167.77	111.18	104.16	180.37	111.18	110.18
	F/U_d%	6.50	6.00	5.50	15.00	6.00	5.50		F/U_d%	5.50	6.33	6.33	8.50	6.33	5.33
B.1.30	C($)	103.99	60.20	51.30	120.31	60.20	59.79	B.1.50	C($)	154.10	100.41	99.71	174.73	100.41	99.69
	F/U_d%	9.00	6.50	6.50	11.00	6.50	5.50		F/U_d%	6.33	6.00	5.67	10.00	6.00	5.67
B.2.30	C($)	111.18	66.80	57.93	138.67	66.80	61.86	B.2.50	C($)	173.74	101.15	100.42	188.97	101.15	100.53
	F/U_d%	5.00	4.33	4.00	11.00	4.33	4.50		F/U_d%	7.50	6.33	6.00	9.00	6.33	5.67
Avg.	C									**138.39**	**87.20**	**82.37**	**153.48**	**87.20**	**85.96**
	F/U_d									**6.54**	**5.81**	**5.54**	**10.56**	**5.81**	**5.19**

5.5.1. Impact of demand proportions

To assess the performance of the algorithm under different demand patterns, we investigate the impact of changes in the proportions of paired, pickup-only and delivery-only demands with instances A/B/C.#.30. Table 15 compares the best costs and average run times for different demand proportions among 5 runs. Overall, the cost of each instance with different proportions of customer nodes is very similar, indicating that the ILNS can still maintain effectiveness when the proportion of customers changes. In particular, the cost slowly decreases as more customers are paired (the smaller ones appear in bold), which is because although each pair of services must be performed by the same vehicle, capacity restrictions on deployed vehicles are somewhat relaxed. In addition, the complexity of the mPDPD-mT increases significantly as more pair services are involved, as a distinct increase in computational times is observed for all instances considered, especially in the instances with wider time windows (in scenario C). In

Table 15
Impact of demand proportions on mPDPD-mT.

Instance	C ($)	$\overline{time}$ (s)	C ($)	$\overline{time}$ (s)	C ($)	$\overline{time}$ (s)	C ($)	$\overline{time}$ (s)	C ($)	$\overline{time}$ (s)	C ($)	$\overline{time}$ (s)
	0-35-65		**10**-30-60		**20**-25-55		**30**-20-50		**40**-15-45		**50**-10-40	
A.1.30	73.71	633.65	73.68	689.13	73.61	1253.06	73.43	1644.40	73.12	1838.42	**72.35**	2145.30
A.2.30	64.47	702.93	64.44	770.23	64.44	1421.06	64.44	1965.94	64.42	2160.76	**64.19**	2581.92
A.3.30	78.41	577.98	73.43	581.72	72.85	687.23	72.44	908.77	72.39	1153.27	**72.31**	1430.92
A.4.30	69.05	1073.60	68.59	1112.25	68.41	1803.20	68.34	2115.55	68.27	2351.01	**68.10**	2851.18
B.1.30	60.24	1083.04	60.20	1225.04	60.14	1620.35	60.03	1961.62	59.99	2205.36	**59.68**	3082.44
B.2.30	66.80	848.79	66.77	1025.16	66.75	1212.08	66.52	1452.06	66.23	1852.62	**65.87**	2143.96
B.3.30	76.87	926.69	75.61	1251.63	75.59	1451.03	75.47	2328.02	**75.35**	2364.10	**75.35**	2692.75
B.4.30	69.23	949.47	69.09	952.36	69.01	1004.20	68.95	1607.83	68.63	2022.64	**68.39**	2843.26
C.1.30	62.49	1404.77	62.37	1752.04	62.39	1872.53	62.35	3021.94	62.20	4536.12	**61.76**	4646.81
C.2.30	62.85	1141.75	62.76	1253.06	62.54	1842.06	62.31	3964.75	62.20	4263.71	**62.05**	4652.09
C.3.30	70.16	1441.96	70.08	1928.62	69.89	2106.39	69.82	2859.05	69.74	3653.26	**69.28**	4002.36
C.4.30	61.86	1955.23	61.61	1959.72	60.89	2145.23	60.55	3431.96	60.34	4591.14	**59.97**	4981.68
Avg.	**68.01**	**1061.66**	**67.39**	**1208.41**	**67.21**	**1534.87**	**67.05**	**2271.82**	**66.91**	**2749.37**	**66.61**	**3171.22**
	10-**0**-90		10-**10**-80		10-**20**-70		10-**30**-60		10-**40**-50		10-**50**-40	
A.1.30	73.81	730.24	72.50	716.48	72.50	701.60	72.48	681.62	72.43	1178.33	**72.36**	825.74
A.2.30	79.24	830.03	74.90	895.31	74.90	777.44	74.89	770.23	**74.84**	1081.91	74.88	923.90
A.3.30	78.74	611.11	73.58	460.44	73.58	594.42	73.43	581.72	**72.74**	807.47	72.80	746.09
A.4.30	72.24	1210.31	69.53	1280.51	68.66	1085.69	68.59	1112.25	68.57	1215.05	**68.32**	1293.56
B.1.30	60.56	974.95	60.37	1032.10	60.31	919.20	60.20	903.49	**60.09**	1611.33	60.18	1197.13
B.2.30	67.34	739.21	67.27	904.51	67.24	1043.02	66.80	1170.29	66.78	978.53	**66.61**	868.28
B.3.30	78.00	918.18	77.27	944.39	76.20	1031.74	75.61	1134.07	75.60	1120.82	**75.33**	1040.20
B.4.30	69.53	739.54	69.13	764.55	69.09	897.45	69.09	840.86	68.99	1047.08	**68.98**	898.99
C.1.30	64.45	1580.93	63.49	1470.95	63.48	1338.73	62.87	1609.31	62.77	1517.45	**62.67**	1502.47
C.2.30	73.18	892.83	72.13	1073.39	71.48	819.11	68.71	1117.24	**68.61**	1230.31	71.40	1218.32
C.3.30	70.83	963.56	70.44	1293.02	70.32	1030.21	70.28	1299.82	69.45	1526.83	**69.37**	1361.09
C.4.30	62.10	1831.98	61.73	1491.11	61.68	1541.87	58.85	1600.70	58.81	1814.30	**58.79**	1665.84
Avg.	**70.84**	**1001.91**	**69.36**	**1027.23**	**69.12**	**1031.71**	**68.48**	**1168.47**	**68.31**	**1260.78**	**68.47**	**1128.47**
	10-90-**0**		10-80-**10**		10-70-**20**		10-60-**30**		10-50-**40**		10-40-**50**	
A.1.30	73.33	830.45	73.24	1025.53	72.51	974.48	72.50	749.00	**72.36**	825.74	72.43	1178.33
A.2.30	77.92	748.78	77.85	1142.81	77.73	1272.92	75.24	808.19	74.88	923.90	**74.84**	1081.91
A.3.30	78.59	639.21	78.15	719.33	75.09	897.27	73.56	629.00	72.80	746.09	**72.74**	807.47
A.4.30	73.27	1252.79	72.81	1690.35	70.55	1512.11	68.98	1180.64	**68.32**	1293.56	68.57	1215.05
B.1.30	60.70	1551.97	60.66	1378.11	60.60	1864.45	60.58	979.98	60.18	1197.13	**60.09**	1611.33
B.2.30	67.20	1160.27	67.20	1039.67	67.13	1146.39	**66.51**	1343.40	66.61	868.28	66.78	978.53
B.3.30	77.09	736.23	76.07	1444.25	76.02	1519.39	76.00	1119.65	**75.33**	1040.20	75.60	1120.82
B.4.30	69.15	857.26	69.15	1147.71	69.14	1442.67	69.00	774.34	**68.98**	898.99	68.99	1047.08
C.1.30	63.72	1291.62	63.57	1558.44	63.35	1735.53	62.82	1626.21	**62.67**	1502.47	62.77	1517.45
C.2.30	72.76	1085.63	72.57	1589.55	72.55	1378.90	72.47	923.51	71.40	1218.32	**68.61**	1230.31
C.3.30	69.95	1795.44	69.77	1485.32	69.63	1578.61	**68.61**	1671.44	69.37	1361.09	69.45	1526.83
C.4.30	61.84	1670.33	61.74	1899.94	61.48	1711.16	60.36	1809.15	**58.79**	1665.84	58.81	1814.30
Avg.	**70.46**	**1135.00**	**70.23**	**1343.42**	**69.65**	**1419.49**	**68.89**	**1134.54**	**68.47**	**1128.47**	**68.31**	**1260.78**

contrast, the solving times required for different proportions of pickup-only and delivery-only instances are relatively similar (within 2000 s). In addition, the cost decreases slightly in most instances as more pickup-only services are involved when the pickup-only proportion is less than 40% or 50%, and the cost begins to rise slowly if otherwise, indicating that providing similar proportions of pickup and delivery services can reduce the cost to some extent. We reason that it can increase vehicle utilisation and weaken their departure capacity constraint from the depot.

Summary. The demand proportion has less impact on the cost of the parallel mode. Pair service significantly increases problem complexity, but this is not the case for pickup-only and delivery-only services. Increasing pair services or providing similar numbers of pickup and delivery services can slightly reduce the total cost.

5.5.2. Impact of node distributions

Given the presumed uniform distribution attributes, in this section a set of instances, each with 30 customers, is designed to further examine the impact of the node distributions. As proposed in Nguyen et al. (2022), the distribution of customers (cd) follows three patterns: random (r), clustered (c), or random-clustered (rc), and three types of depot position settings (dp) are considered, namely, central (c), edge (e) and random (r), located at the centre, edge, and a random position of the grid, respectively. The type of time windows (tw) and the customer demands are generated as in Section 5.1. In addition, three levels of time window width (wd) are considered, 30, 60 and 90 (unit: min), for comparison.

Table 16 displays the best total costs for the truck-only mode and the mPDPD-mT among 5 runs and the corresponding ratio of the number of drone-served customers to that of deployed trips (R/F). Overall, the total costs of both modes decrease as the time

Table 16
Impact of node distributions on mPDPD-mT.

Instance			wd = 20 min				wd = 60 min				wd = 90 min			
tw	cd	dp	C_t ($)	C ($)	R/F%	Δ_{c,c_t}%	C_t ($)	C ($)	R/F%	Δ_{c,c_t}%	C_t ($)	C ($)	R/F%	Δ_{c,c_t}%
A	c	c	151.16	76.82	1.92	−49.18	127.31	68.94	2.18	−45.85	119.62	67.72	2.67	−43.39
A	c	e	172.73	126.81	1.64	−26.58	141.40	82.46	1.90	−41.68	133.55	79.71	2.25	−40.31
A	c	r	155.35	79.02	1.79	−49.13	128.28	71.37	2.00	−44.36	118.72	67.41	2.30	−43.22
A	r	c	191.00	81.92	1.67	−57.11	156.17	74.25	2.09	−52.46	149.37	71.96	2.17	−51.82
A	r	e	205.88	129.43	1.50	−37.13	160.34	121.28	1.70	−24.36	154.95	95.25	1.77	−38.53
A	r	r	193.90	104.12	1.53	−46.30	154.24	88.22	1.68	−42.80	151.51	74.83	1.78	−50.61
A	rc	c	162.73	57.83	1.69	−64.46	137.89	50.34	2.25	−63.49	126.06	49.19	2.70	−60.98
A	rc	e	177.62	133.57	1.68	−24.80	151.54	104.34	1.83	−31.15	139.43	83.00	1.88	−40.47
A	rc	r	159.72	61.37	1.69	−61.58	139.17	52.90	2.17	−61.99	125.81	52.23	2.36	−58.49
Avg.					**1.68**	**−46.25**			**1.98**	**−45.35**			**2.21**	**−47.54**
B	c	c	152.41	58.66	1.92	−61.51	127.31	50.56	2.22	−60.29	119.62	49.71	2.50	−58.44
B	c	e	181.53	129.64	1.71	−28.58	141.40	114.87	1.70	−18.76	133.55	113.97	1.80	−14.66
B	c	r	156.96	58.12	1.75	−62.97	128.28	50.22	2.22	−60.85	118.72	49.48	2.25	−58.32
B	r	c	201.91	74.98	1.38	−62.86	156.17	71.77	2.00	−54.04	149.37	65.99	2.00	−55.82
B	r	e	211.14	125.05	1.29	−40.77	160.34	102.16	1.50	−36.29	154.95	95.83	1.55	−38.15
B	r	r	206.97	88.71	1.50	−57.14	154.24	72.14	1.91	−53.23	151.51	70.52	2.22	−53.46
B	rc	c	165.26	59.85	2.00	−63.78	137.89	58.24	2.27	−57.76	126.06	56.95	2.78	−54.82
B	rc	e	181.86	132.32	1.71	−27.24	151.54	100.01	1.71	−34.00	139.43	98.23	1.83	−29.55
B	rc	r	163.09	66.12	1.85	−59.46	139.17	62.80	2.44	−54.88	125.81	57.27	3.29	−54.48
Avg.					**1.68**	**−51.59**			**2.00**	**−47.79**			**2.25**	**−46.41**

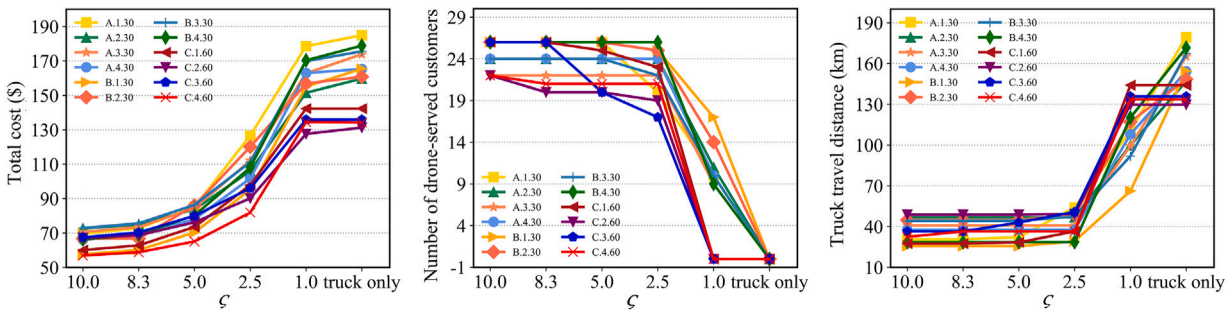

Fig. 5. Impact of vehicle cost ratio on mPDPD-mT.

window widens, and more customers can be served within a trip, as described in Section 5.4.1. Moreover, the truck-only mode is more sensitive to the width of time windows. The cost reduction compared with the truck-only mode is more than 40%, which is in line with the results in Section 5.2.2, and shows almost the same pattern between the two types of time windows, highlighting the robust performance of the proposed ILNS in different location distributions. Specifically, we observe that the customer distribution seems to have less impact on the final results, whereas the location of the depot heavily influences both modes, especially when the time window is tight. In particular, the cases with the depot located at the centre usually have the lowest cost, while the cost of the cases with the depot located at the edge is typically the highest, and the R/F ratio and cost savings over the truck-only mode are the lowest.

Summary. The width of the time windows and node distribution, especially the location of the depot, have a significant effect on the parallel mode. Widening time windows or Locating the depot in the centre instead of at the edge of the grid leads to the highest cost savings compared to the truck-only mode, and more customers can be served in a trip.

5.5.3. Impact of drone cost

This section investigates the impact of the vehicle cost ratio (ς) on instances A/B/C.#.30 in terms of total cost, number of drone-served customers and distance travelled by trucks, as detailed in Fig. 5. These results indicate that as the drone unit cost increases, that is, when ς decreases (the truck unit cost is assumed to be fixed), the total cost increases, drones serve fewer customers, and therefore, more distance is travelled by trucks accordingly. Moreover, these changes are more obvious when ς is smaller than 2.5, before which the increase or reduction rate is relatively small. We reason that if the unit cost of drones remains low ($\varsigma > 2.5$), the benefits of the parallel mode will become more evident (as pointed out in Section 5.2.4). However, even if $\varsigma = 1.0$, the parallel mode is still advantageous over the truck-only mode, especially in instances with tighter time windows (in scenarios A and B). Moreover, scenarios A and B consume higher costs than scenario C for the same ς; however, there are no clear differences in the truck travel distance for most cases. Another observation is that scenario C is more sensitive to ς than the other scenarios are.

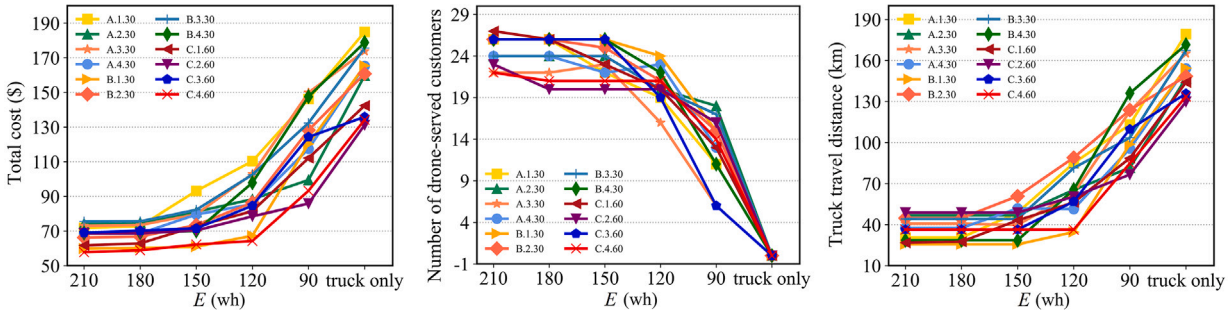

Fig. 6. Impact of drone battery capacity on mPDPD-mT.

Summary. The benefits of the parallel mode are more sensitive to drone cost when the time window is wide or the drone unit cost is relatively high (the vehicle cost ratio is less than 2.5). Even if their costs are the same, the advantage over the truck-only mode still exists when the time window is tight.

5.5.4. Impact of drone battery capacity

Considering the significant impact of drone battery capacity (E) as previously described, we explore its impacts on items similar to those in the previous section. The detailed results of instances A/B/C.#.30 provided by the ILNS are shown in Fig. 6. In general, as the drone battery capacity decreases, the total cost gradually increases, drones serve fewer customers, and therefore more distance is travelled by trucks accordingly. Moreover, as long as E remains above 120 Wh, the values of the three quantities are relatively stable, and drone services are available for approximately 70% of the drone-eligible customers. We can infer that the parallel mode is preferable if the drone battery capacity is maintained at a relatively high level. However, even if E decreases to 90, the parallel mode still has clear advantages over the truck-only mode for all considered instances.

Summary. As long as the drone battery capacity (E) is maintained at a certain level, such as above 120 Wh (a fully loaded drone can operate for 20 min), the benefits of the parallel mode are relatively stable. Even if the capacity drops to half of the baseline (15 min for a fully loaded drone), the advantage is still significant, with a cost savings of approximately 25% compared with the truck-only mode.

6. Conclusions

This paper has investigated a new PDSVRP variant for last-mile logistics, namely, the multi-visit parallel drone scheduling pickup and delivery problem considering multiple trips and time windows (mPDPD-mT). In this problem, multiple available, homogeneous trucks and drones operate in parallel to serve customers with pick-up and delivery requests. Some customers, namely, customer pairs, are paired and have one-to-one correspondence. Each drone departs and returns from the depot, serving one or more customers en route, and can conduct multiple trips. Each customer has a time window within which the service can be started, assuming that drones can wait at the depot to save energy from hovering over customer locations because of early arrival before time windows.

With the goal of minimising total cost, we propose a mixed-integer bilinear program with several strengthening inequalities, which is then solved via a custom metaheuristic algorithm called ILNS. Moreover, an effective approach has been developed to optimise drone waiting time at the depot during each trip. Finally, multiple extensive experiments were conducted to validate our MIBLP and demonstrate the performance of the proposed ILNS. Overall, the results of test instances with up to 200 customers highlight that our ILNS can yield high-quality solutions in a reasonable time and dominate the benchmark algorithms available from the literature, with a total cost reduction of 5% on average. Moreover, we thoroughly evaluated the effectiveness of the waiting time optimisation approach and performed a series of sensitivity analyses to explore the problem features quantitatively and the impacts of key parameters on the results of the parallel system. The results of numerical experiments on various instances indicate that optimising drone waiting times can substantially reduce the total cost, and it would be preferable to allow multiple trips and multiple visits of drones within reasonable limits. Allowing multiple visits, especially multiple flights, can significantly reduce the total cost (by 15% and 65%), although their performances are impacted to different extents by several key parameters. In terms of drone parameters, as long as the truck-drone cost ratio is above 2 or the drone battery capacity is maintained above 120 Wh (2/3 of the baseline), the benefits of the parallel mode can be prominent and relatively stable, and even if the battery capacity drops to half of the baseline (15 min for a fully loaded drone), the advantage is still significant, with a cost savings of approximately 25% over the truck-only mode. For node features, the time window width and customer distribution, especially the location of the depot, are key to maintaining attractive benefits in parallel operation. Locating the depot in the centre or widening time windows leads to the lowest cost, and more customers can be served in a trip. Parameters such as demand proportions have little effect on the final results. Increasing pair services or providing similar numbers of pickup and delivery services can slightly reduce costs.

Owing to several potential barriers – regulations, infrastructure, and technology – remaining immature for the development of drone transport, drone deliveries have yet to be widely adopted in practice, and commercial drone adoptions remain at a small

scale. Notwithstanding these hurdles, the academic community and industry alike are optimistic about the wider adoption of drone deliveries, as mentioned in the Introduction. As this approach continues to evolve, it is not surprising that drone adoption could be widely materialised in the foreseeable future.

Building upon this work, future research may include variants that take in other additional attributes, such as using heterogeneous types of both vehicles or allowing multi-depot usage to expand the service coverage. Given the significant complexity of the problem, further scaling up experiments and devising more effective algorithms to be tractable for much larger problems could be a very interesting extension. Another interesting issue is incorporating relevant practical applications or real-life case studies to increase the credibility of the research and provide more valuable insights in the field. Finally, the research direction can be a dynamic scenario in which customer requests dynamically appear or a stochastic version appears with some uncertain factors, such as stochastic drone energy consumption due to unpredictable conditions.

Declaration of competing interest

The author declares that she has no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work is supported by the China Scholarship Council [grant no. 202207000043] and the Doctoral Start-up Fund Project of Shanghai Jian Qiao University [grant no. KYJF24BBYQ202403].

Appendix A. Supplementary material

Supplementary material related to this article can be found online at https://doi.org/10.1016/j.tre.2025.104351.

References

Agatz, N., Bouman, P., Schmidt, M., 2018. Optimization approaches for the traveling salesman problem with drone. Transp. Sci. 52 (4), 965–981.
Avci, M., Topaloglu, S., 2016. A hybrid metaheuristic algorithm for heterogeneous vehicle routing problem with simultaneous pickup and delivery. Expert Syst. Appl. 53, 160–171.
Chen, X., Ulmer, M.W., Thomas, B.W., 2022. Deep Q-learning for same-day delivery with vehicles and drones. European J. Oper. Res. 298 (3), 939–952.
Chen, P., Wang, Q., 2024. Learning for multiple purposes: A Q-learning enhanced hybrid metaheuristic for parallel drone scheduling traveling salesman problem. Comput. Ind. Eng. 187, 109851.
Dell'Amico, M., Montemanni, R., Novellani, S., 2020. Matheuristic algorithms for the parallel drone scheduling traveling salesman problem. Ann. Oper. Res. 289, 211–226.
Dinh, Q.T., Do, D.D., Hà, M.H., 2021. Ants can solve the parallel drone scheduling traveling salesman problem. In: Proceedings of the Genetic and Evolutionary Computation Conference. pp. 14–21.
DJI, 2025. DJI FlyCart 30. URL https://www.dji.com/cn/flycart-30.
Flavin, P., 2022. Drone logistics: How drone tech is shaking up supply chains. URL https://www.rasmussen.edu/degrees/business/blog/drone-logistics/.
Ham, A.M., 2018. Integrated scheduling of m-truck, m-drone, and m-depot constrained by time-window, drop-pickup, and m-visit using constraint programming. Transp. Res. Part C: Emerg. Technol. 91, 1–14.
Jiang, J., Dai, Y., Yang, F., Ma, Z., 2024. A multi-visit flexible-docking vehicle routing problem with drones for simultaneous pickup and delivery services. European J. Oper. Res. 312 (1), 125–137.
Karak, A., Abdelghany, K., 2019. The hybrid vehicle-drone routing problem for pick-up and delivery services. Transp. Res. Part C: Emerg. Technol. 102, 427–449.
Keçeci, B., Altıparmak, F., Kara, İ., 2021. A mathematical formulation and heuristic approach for the heterogeneous fixed fleet vehicle routing problem with simultaneous pickup and delivery. J. Ind. Manag. Optim. 17 (3), 1069–1100.
Kitjacharoenchai, P., Min, B.-C., Lee, S., 2020. Two echelon vehicle routing problem with drones in last mile delivery. Int. J. Prod. Econ. 225, 107598.
Koç, Ç., Laporte, G., Tükenmez, İ., 2020. A review of vehicle routing with simultaneous pickup and delivery. Comput. Oper. Res. 122, 104987.
Lei, D., Chen, X., 2022. An improved variable neighborhood search for parallel drone scheduling traveling salesman problem. Appl. Soft Comput. 127, 109416.
Li, D., Ignatius, J., Wang, D., Yin, Y., Cheng, T., 2024. A branch-and-price-and-cut algorithm for the truck-drone routing problem with simultaneously delivery and pickup. Naval Res. Logist. 71 (2), 241–285.
Liu, Y., Liu, Z., Shi, J., Wu, G., Pedrycz, W., 2020. Two-echelon routing problem for parcel delivery by cooperated truck and drone. IEEE Trans. Syst. Man Cybern. Syst. 51 (12), 7450–7465.
Luo, Z., Gu, R., Poon, M., Liu, Z., Lim, A., 2022. A last-mile drone-assisted one-to-one pickup and delivery problem with multi-visit drone trips. Comput. Oper. Res. 148, 106015.
Madankumar, S., Rajendran, C., 2019. A mixed integer linear programming model for the vehicle routing problem with simultaneous delivery and pickup by heterogeneous vehicles, and constrained by time windows. Sādhanā 44, 1–14.
Masmoudi, M.A., Mancini, S., Baldacci, R., Kuo, Y.-H., 2022. Vehicle routing problems with drones equipped with multi-package payload compartments. Transp. Res. Part E: Logist. Transp. Rev. 164, 102757.
Mbiadou Saleu, R.G., Deroussi, L., Feillet, D., Grangeon, N., Quilliot, A., 2018. An iterative two-step heuristic for the parallel drone scheduling traveling salesman problem. Netw. 72 (4), 459–474.
Meng, S., Chen, Y., Li, D., 2024. The multi-visit drone-assisted pickup and delivery problem with time windows. European J. Oper. Res. 314 (2), 685–702.
Meng, S., Guo, X., Li, D., Liu, G., 2023. The multi-visit drone routing problem for pickup and delivery services. Transp. Res. Part E: Logist. Transp. Rev. 169, 102990.
Montemanni, R., Dell'Amico, M., 2023. Constraint programming models for the parallel drone scheduling vehicle routing problem. EURO J. Comput. Optim. 11, 100078.
Montemanni, R., Dell'Amico, M., Corsini, A., 2024. Parallel drone scheduling vehicle routing problems with collective drones. Comput. Oper. Res. 163, 106514.
Murray, C.C., Chu, A.G., 2015. The flying sidekick traveling salesman problem: Optimization of drone-assisted parcel delivery. Transp. Res. Part C: Emerg. Technol. 54, 86–109.

Nepomuceno, N., Barboza Saboia, R., Rogério Pinheiro, P., 2019. A fast randomized algorithm for the heterogeneous vehicle routing problem with simultaneous pickup and delivery. Algorithms 12 (8), 158.

Nguyen, M.A., Dang, G.T.-H., Hà, M.H., Pham, M.-T., 2022. The min-cost parallel drone scheduling vehicle routing problem. European J. Oper. Res. 299 (3), 910–930.

Nguyen, M.A., Hà, M.H., 2023. The parallel drone scheduling traveling salesman problem with collective drones. Transp. Sci. 57 (4), 866–888.

Nguyen, M.A., Luong, H.L., Hà, M.H., Ban, H.-B., 2023. An efficient branch-and-cut algorithm for the parallel drone scheduling traveling salesman problem. 4OR 21 (4), 609–637.

Qu, Y., Bard, J.F., 2015. A branch-and-price-and-cut algorithm for heterogeneous pickup and delivery problems with configurable vehicle capacity. Transp. Sci. 49 (2), 254–270.

Salama, M., Srinivas, S., 2020. Joint optimization of customer location clustering and drone-based routing for last-mile deliveries. Transp. Res. Part C: Emerg. Technol. 114, 620–642.

Saleu, R.G.M., Deroussi, L., Feillet, D., Grangeon, N., Quilliot, A., 2022. The parallel drone scheduling problem with multiple drones and vehicles. European J. Oper. Res. 300 (2), 571–589.

Shakir, U., 2023. Wing's new delivery network lets its drones deliver more with fewer stops. URL https://www.theverge.com/2023/3/9/23631068/wing-delivery-network-drone-autoloader.

Shaw, P., 1997. A new local search algorithm providing high quality solutions to vehicle routing problems. APES Group Dept. Comput. Sci. Univ. Strat. Glas. Scotl. 46.

Simmons, R., 2023. LAST mile delivery: Solutions for your business. URL https://www.dhl.com/discover/en-my/logistics-advice/import-export-advice/last-mile-solutions.

Tong, B., Wang, J., Wang, X., Zhou, F., Mao, X., Zheng, W., 2022. Optimal route planning for truck–Drone delivery using variable neighborhood tabu search algorithm. Appl. Sci. 12 (1), 529.

Wang, D., Hu, P., Du, J., Zhou, P., Deng, T., Hu, M., 2019. Routing and scheduling for hybrid truck-drone collaborative parcel delivery with independent and truck-carried drones. IEEE Internet Things J. 6 (6), 10483–10495.

Wang, X., Poikonen, S., Golden, B., 2017. The vehicle routing problem with drones: Several worst-case results. Optim. Lett. 11 (4), 679–697.

Wing, 2025. How wing's drone delivery technology works. URL https://wing.com/technology.

Wolfinger, D., 2021. A large neighborhood search for the pickup and delivery problem with time windows, split loads and transshipments. Comput. Oper. Res. 126, 105110.

Zhen, L., Gao, J., Tan, Z., Wang, S., Baldacci, R., 2023. Branch-price-and-cut for trucks and drones cooperative delivery. IISE Trans. 55 (3), 271–287.

Zipline, 2025. Deliver food to your custoners by drone in under 10 min. URL https://www.flyzipline.com/solutions/restaurants.